
FitBuddy > app > __init__.py
1
Explain and Fix Problem (Ctrl+Shift+.)
y-complete\FitBuddy\app\config.py


FitBuddy > app > __init__.py
1
Explain and Fix Problem (Ctrl+Shift+.)
y-complete\FitBuddy\app\config.py


FitBuddy > app > __init__.py
1
Explain and Fix Problem (Ctrl+Shift+.)
y-complete\FitBuddy\app\config.py

FitBuddy > app > config.py > ...

```python
import os
from dotenv import load_dotenv
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent
load_dotenv(BASE_DIR / ".env")

DATABASE_URL = os.getenv("DATABASE_URL", f"sqlite:///{BASE_DIR / 'fitbuddy.db'}")

# Current Google Gen AI SDK model names can be changed here without editing application code.
GEMINI_WORKOUT_MODEL = os.getenv("GEMINI_WORKOUT_MODEL", "gemini-2.5-flash")
GEMINI_TIP_MODEL = os.getenv("GEMINI_TIP_MODEL", "gemini-2.5-flash")
GOOGLE_API_KEY = os.getenv("GOOGLE_API_KEY", "").strip()

ADMIN_KEY = os.getenv("ADMIN_KEY", "fitbuddy-admin")

```

ddy-complete\FitBuddy\app\database.py

FitBuddy > app > database.py

```python
Base = declarative_base()

class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True, index=True)
    user_id = Column(String(100), unique=True, nullable=False, index=True)
    username = Column(String(120), nullable=False)
    age = Column(Integer, nullable=False)
    weight = Column(Float, nullable=False)
    goal = Column(String(100), nullable=False)
    intensity = Column(String(20), nullable=False)
    created_at = Column(DateTime, default=datetime.utcnow)
    plans = relationship("Plan", back_populates="user", cascade="all, delete-orphan")

class Plan(Base):
    __tablename__ = "plans"
    id = Column(Integer, primary_key=True, index=True)
    user_id = Column(Integer, ForeignKey("users.id"), nullable=False)
    original_plan = Column(Text, nullable=False)
```

FitBuddy > app > database.py > ...

```
11  class User(Base):
18      goal = Column(String(100), nullable=False)
19      intensity = Column(String(20), nullable=False)
20      created_at = Column(DateTime, default=datetime.utcnow)
21      plans = relationship("Plan", back_populates="user", cascade="all, delete-orphan")
22
23  class Plan(Base):
24      __tablename__ = "plans"
25      id = Column(Integer, primary_key=True, index=True)
26      user_id = Column(Integer, ForeignKey("users.id"), nullable=False)
27      original_plan = Column(Text, nullable=False)
28      updated_plan = Column(Text, nullable=True)
29      nutrition_tip = Column(Text, nullable=True)
30      feedback = Column(Text, nullable=True)
31      created_at = Column(DateTime, default=datetime.utcnow)
32      updated_at = Column(DateTime, nullable=True)
33      user = relationship("User", back_populates="plans")
34
35  def init_db():
```

FitBuddy > app > database.py > ...

```
11  class User(Base):
20      created_at = Column(DateTime, default=datetime.utcnow)
21      plans = relationship("Plan", back_populates="user", cascade="all, delete-orphan")
22
23  class Plan(Base):
24      __tablename__ = "plans"
25      id = Column(Integer, primary_key=True, index=True)
26      user_id = Column(Integer, ForeignKey("users.id"), nullable=False)
27      original_plan = Column(Text, nullable=False)
28      updated_plan = Column(Text, nullable=True)
29      nutrition_tip = Column(Text, nullable=True)
30      feedback = Column(Text, nullable=True)
31      created_at = Column(DateTime, default=datetime.utcnow)
32      updated_at = Column(DateTime, nullable=True)
33      user = relationship("User", back_populates="plans")
34
35  def init_db():
36      Base.metadata.create_all(bind=engine)
37
38  def save_user(data):
```

FitBuddy > app > database.py > ...

```python
def save_user(data):
    db = SessionLocal()
    try:
        user = db.query(User).filter(User.user_id == data.user_id).first()
        if user:
            user.username = data.username
            user.age = data.age
            user.weight = data.weight
            user.goal = data.goal
            user.intensity = data.intensity
        else:
            user = User(
                user_id=data.user_id, username=data.username, age=data.age,
                weight=data.weight, goal=data.goal, intensity=data.intensity
            )
            db.add(user)
        db.commit()
        db.refresh(user)
        return user
```

FitBuddy > app > database.py > ...

```
38    def save_user(data):
42            if user:
43                user.username = data.username
44                user.age = data.age
45                user.weight = data.weight
46                user.goal = data.goal
47                user.intensity = data.intensity
48            else:
49                user = User(
50                    user_id=data.user_id, username=data.username, age=data.age,
51                    weight=data.weight, goal=data.goal, intensity=data.intensity
52                )
53                db.add(user)
54            db.commit()
55            db.refresh(user)
56            return user
57        finally:
58            db.close()
59
```

FitBuddy > app > database.py > ...

```
38  def save_user(data):
57      finally:
58          db.close()
59
60  def save_plan(user_pk, workout_plan, nutrition_tip):
61      db = SessionLocal()
62      try:
63          plan = Plan(
64              user_id=user_pk,
65              original_plan=workout_plan,
66              nutrition_tip=nutrition_tip
67          )
68          db.add(plan)
69          db.commit()
70          db.refresh(plan)
71          return plan
72      finally:
73          db.close()
74
75  def get_user(user_id):
```

FitBuddy > app > database.py > ...

```
60    def save_plan(user_pk, workout_plan, nutrition_tip):
63            plan = Plan(
66                nutrition_tip=nutrition_tip
67            )
68            db.add(plan)
69            db.commit()
70            db.refresh(plan)
71            return plan
72        finally:
73            db.close()
74
75    def get_user(user_id):
76        db = SessionLocal()
77        try:
78            return db.query(User).filter(User.user_id == user_id).first()
79        finally:
80            db.close()
81
82    def get_latest_plan(user_id):
83        db = SessionLocal()
```

(class) User: def User(...) -> User: ...

Go to User

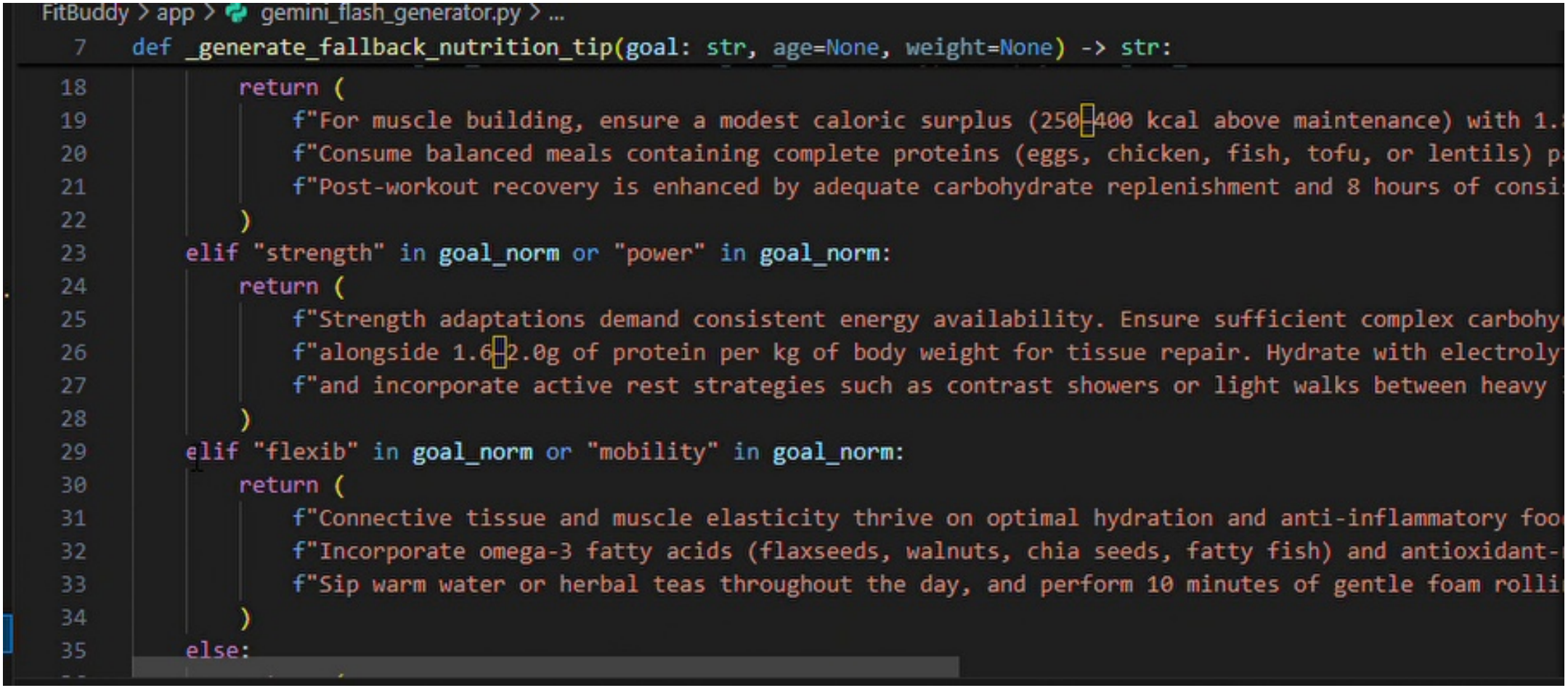
FitBuddy > app > database.py > ...

```
82    def get_latest_plan(user_id):
88            plan = (
90                .filter(Plan.user_id == user.id)
91                .order_by(Plan.id.desc())
92                .first()
93            )
94            return user, plan
95        finally:
96            db.close()
97
98    def update_plan(plan_id, updated_plan, feedback):
99        db = SessionLocal()
100       try:
101           plan = db.query(Plan).filter(Plan.id == plan_id).first()
102           if not plan:
103               return None
104           plan.updated_plan = updated_plan
105           plan.feedback = feedback
106           plan.updated_at = datetime.utcnow()
```

FitBuddy > app > database.py > ...

```
113    def get_all_users_with_plans():
118            db.close()
119
120    def delete_user(user_id):
121        db = SessionLocal()
122        try:
123            user = db.query(User).filter(User.user_id == user_id).first()
124            if user:
125                db.delete(user)
126                db.commit()
127                return True
128            return False
129        finally:
130            db.close()
131
```


FitBuddy > app > database.py > ...
130 db.close()
131


FitBuddy > app > gemini_flash_generator.py > ...
def _generate_fallback_nutrition_tip(goal: str, age=None, weight=None) -> str:
return (
f"For muscle building, ensure a modest caloric surplus (250-400 kcal above maintenance) with 1.
f"Consume balanced meals containing complete proteins (eggs, chicken, fish, tofu, or lentils) p
f"Post-workout recovery is enhanced by adequate carbohydrate replenishment and 8 hours of consi
)
elif "strength" in goal_norm or "power" in goal_norm:
return (
f"Strength adaptations demand consistent energy availability. Ensure sufficient complex carbohy
f"alongside 1.6-2.0g of protein per kg of body weight for tissue repair. Hydrate with electroly
f"and incorporate active rest strategies such as contrast showers or light walks between heavy
)
elif "flexib" in goal_norm or "mobility" in goal_norm:
return (
f"Connective tissue and muscle elasticity thrive on optimal hydration and anti-inflammatory foo
f"Incorporate omega-3 fatty acids (flaxseeds, walnuts, chia seeds, fatty fish) and antioxidant-
f"Sip warm water or herbal teas throughout the day, and perform 10 minutes of gentle foam rolli
)
else:

FitBuddy > app > gemini_flash_generator.py > ...

```python
def _generate_fallback_nutrition_tip(goal: str, age=None, weight=None) -> str:
                f"Connective tissue and muscle elasticity thrive on optimal hydration and anti-inflammatory foo
                f"Incorporate omega-3 fatty acids (flaxseeds, walnuts, chia seeds, fatty fish) and antioxidant-
                f"Sip warm water or herbal teas throughout the day, and perform 10 minutes of gentle foam rolli
            )
        else:
            return (
                f"For sustainable general wellness, focus on balanced whole foods: half your plate with colorfu
                f"and a quarter with complex carbohydrates. Drink water consistently throughout the day (aiming
                f"and prioritize a consistent sleep schedule to allow your muscles and nervous system to rejuve
            )

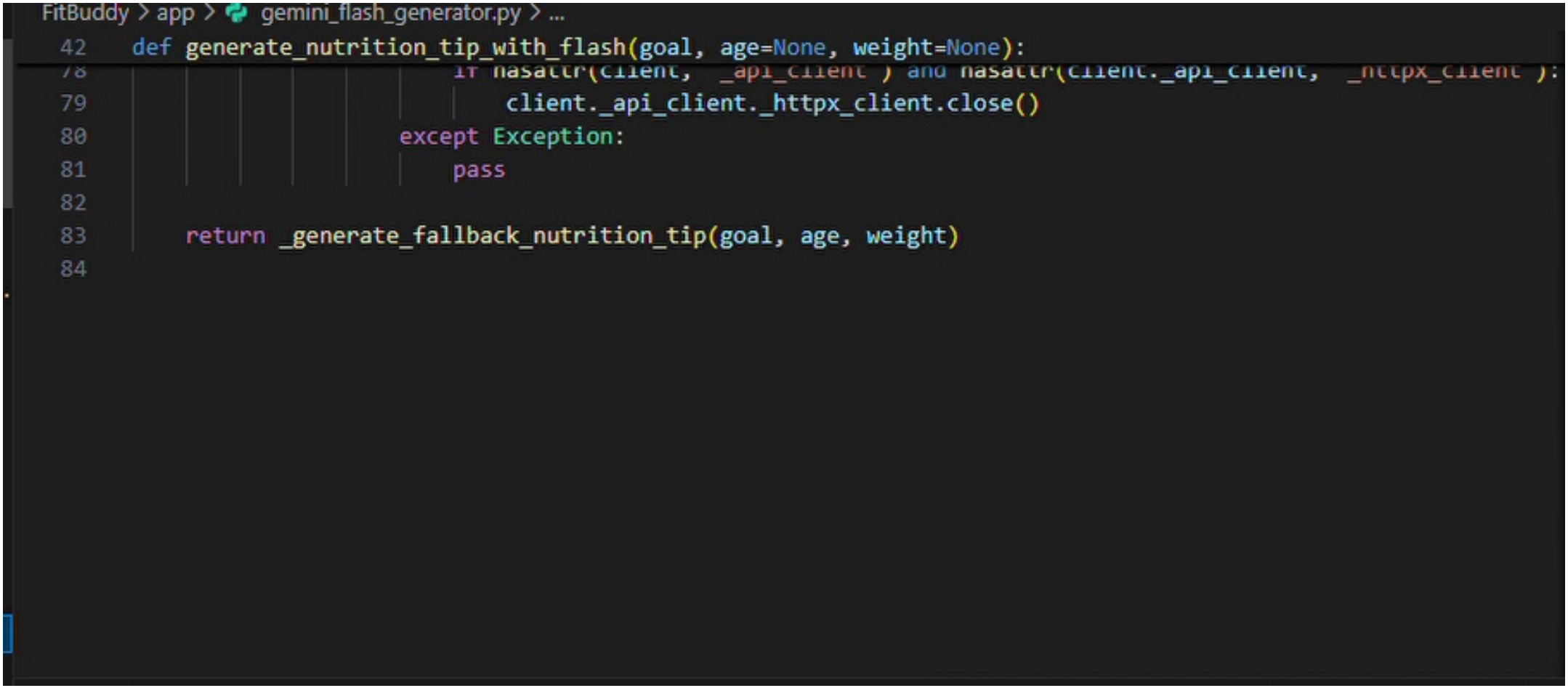
def generate_nutrition_tip_with_flash(goal, age=None, weight=None):
    prompt = f"""
Give one concise, practical nutrition or recovery tip for a person whose fitness goal is "{goal}".
The person is age {age} and weighs {weight} kg.
Keep it general wellness guidance, not a medical prescription.
Mention a practical food, hydration, sleep, or recovery action.
Maximum 120 words.
"""
```

FitBuddy > app > gemini_flash_generator.py > ...

```
41
42  def generate_nutrition_tip_with_flash(goal, age=None, weight=None):
43      prompt = f"""
44  Give one concise, practical nutrition or recovery tip for a person whose fitness goal is "{goal}".
45  The person is age {age} and weighs {weight} kg.
46  Keep it general wellness guidance, not a medical prescription.
47  Mention a practical food, hydration, sleep, or recovery action.
48  Maximum 120 words.
49  """
50      if GOOGLE_API_KEY and not GOOGLE_API_KEY.startswith("AQ."):
51          candidates = [
52              GEMINI_TIP_MODEL,
53              "gemini-2.5-flash",
54              "gemini-3.8-flash",
55              "gemini-1.5-flash",
56              "gemini-flash-latest"
57          ]
58          seen = set()
59          for model_name in candidates:
```

FitBuddy > app > gemini_flash_generator.py > ...

```
42  def generate_nutrition_tip_with_flash(goal, age=None, weight=None):
60              if not model_name or model_name in seen:
61                  continue
62              seen.add(model_name)
63              client = None
64              try:
65                  client = genai.Client(api_key=GOOGLE_API_KEY)
66                  response = client.models.generate_content(
67                      model=model_name,
68                      contents=prompt,
69                  )
70                  if response and response.text and response.text.strip():
71                      return response.text.strip()
72              except Exception as exc:
73                  logger.warning("Gemini tip model %s failed: %s", model_name, exc)
74                  continue
75              finally:
76                  if client is not None:
77                      try:
```


FitBuddy > app > gemini_flash_generator.py > ...
42 def generate_nutrition_tip_with_flash(goal, age=None, weight=None):
79 client._api_client._httpx_client.close()
80 except Exception:
81 pass
83 return _generate_fallback_nutrition_tip(goal, age, weight)

FitBuddy > app > gemini_generator.py > ...

```python
def _call_gemini(prompt: str) -> str:

    candidates = [
        GEMINI_WORKOUT_MODEL,
        "gemini-2.5-flash",
        "gemini-3.8-flash",
        "gemini-1.5-flash",
        "gemini-flash-latest"
    ]
    seen = set()
    last_exc = None

    for model_name in candidates:
        if not model_name or model_name in seen:
            continue
        seen.add(model_name)
        client = None
        try:
            client = genai.Client(api_key=GOOGLE_API_KEY)
```

FitBuddy > app > gemini_generator.py > ...

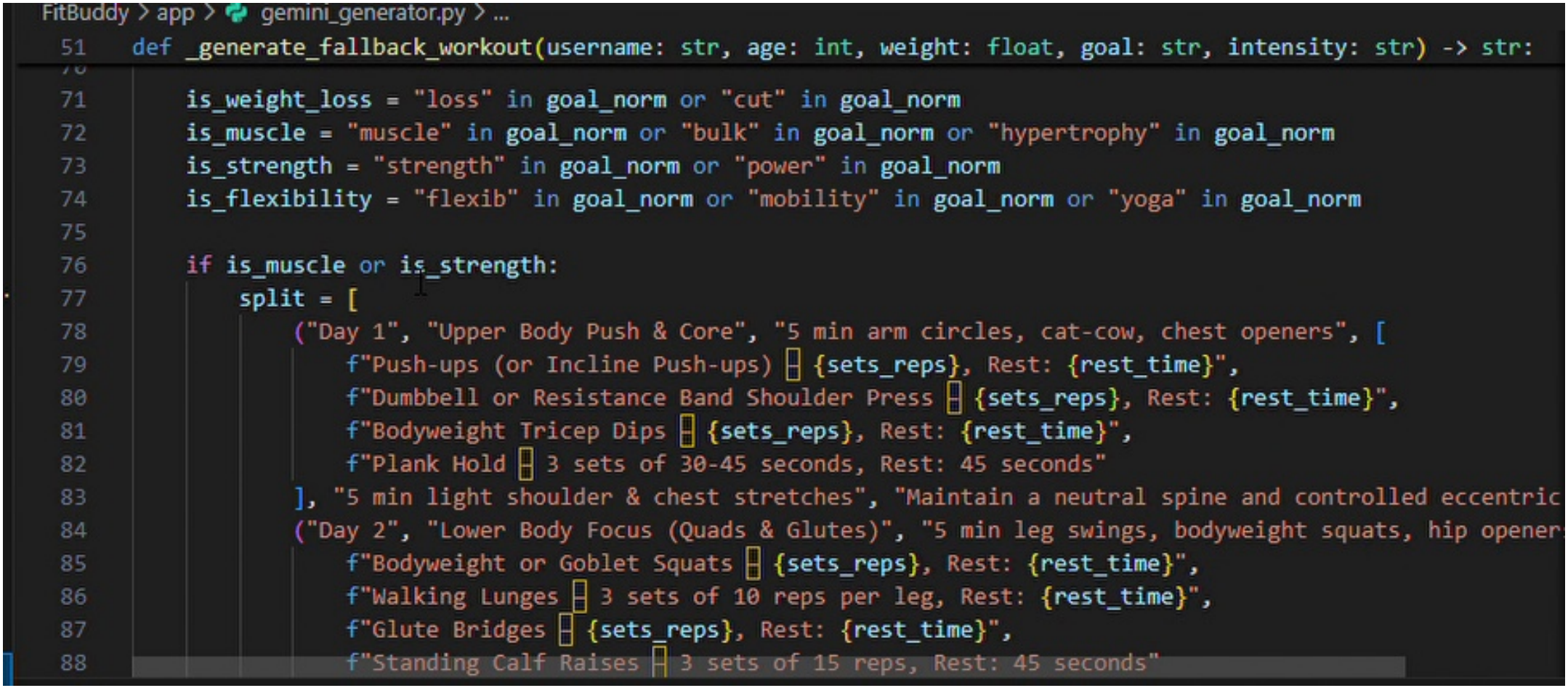
```
7   def _call_gemini(prompt: str) -> str:
26          try:
27              client = genai.Client(api_key=GOOGLE_API_KEY)
28              response = client.models.generate_content(
29                  model=model_name,
30                  contents=prompt,
31              )
32              if response and response.text and response.text.strip():
33                  return response.text.strip()
34          except Exception as exc:
35              last_exc = exc
36              logger.warning("Gemini model %s failed: %s", model_name, exc)
37              continue
38          finally:
39              if client is not None:
40                  try:
41                      if hasattr(client, "_api_client") and hasattr(client._api_client, "_httpx_client"):
42                          client._api_client._httpx_client.close()
43                  except Exception:
```

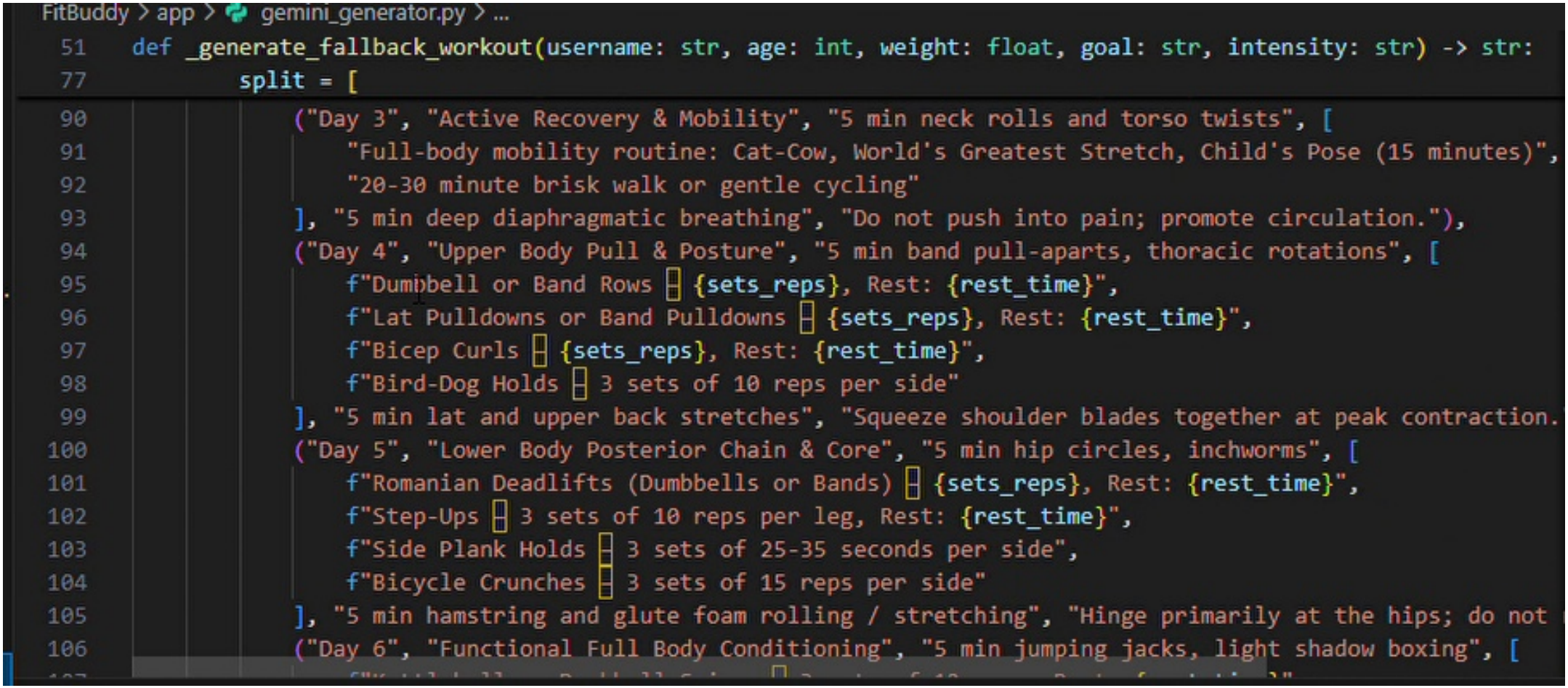
FitBuddy > app > gemini_generator.py > ...

```
7   def _call_gemini(prompt: str) -> str:
36              logger.warning("Gemini model %s failed: %s", model_name, exc)
37              continue
38          finally:
39              if client is not None:
40                  try:
41                      if hasattr(client, "_api_client") and hasattr(client._api_client, "_httpx_client"):
42                          client._api_client._httpx_client.close()
43                  except Exception:
44                      pass
45
46      if last_exc:
47          raise last_exc
48      raise RuntimeError("Failed to generate plan from Gemini API.")
49
50
51  def _generate_fallback_workout(username: str, age: int, weight: float, goal: str, intensity: str) -> str:
52      goal_norm = (goal or "General wellness").lower()
53      intensity_norm = (intensity or "medium").lower()
54
```

FitBuddy > app > gemini_generator.py > ...

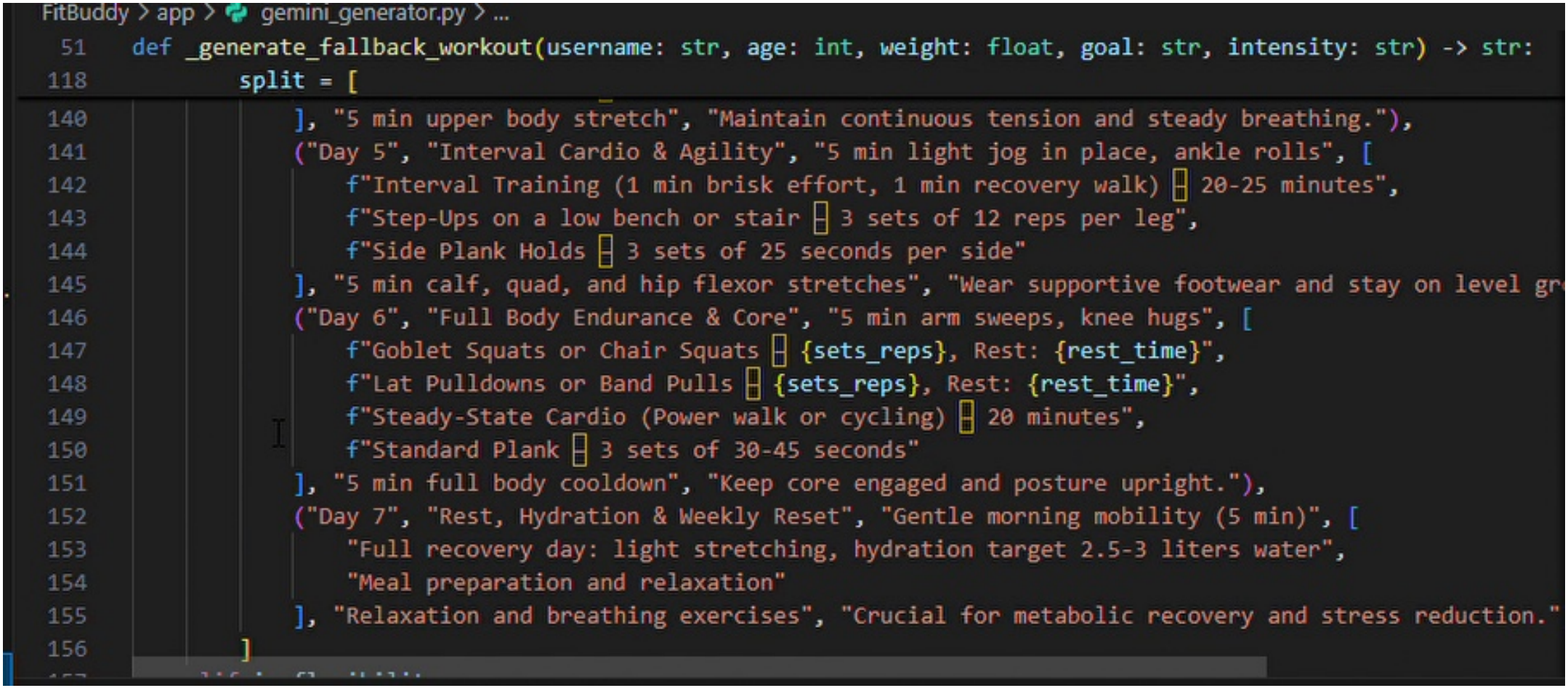
```
51  def _generate_fallback_workout(username: str, age: int, weight: float, goal: str, intensity: str) -> str:
55      if intensity_norm == "low":
56          intensity_desc = "Low-Impact / Gentle Progression"
57          rest_time = "60-90 seconds"
58          cardio_time = "20-25 minutes at an easy, conversational pace"
59          sets_reps = "2-3 sets of 8-10 controlled reps"
60      elif intensity_norm == "high":
61          intensity_desc = "High-Intensity / Progressive Overload"
62          rest_time = "45-60 seconds (circuit) or 2 minutes (heavy lifts)"
63          cardio_time = "25-30 minutes HIIT or high-tempo intervals"
64          sets_reps = "4 sets of 10-15 reps with challenging resistance"
65      else:
66          intensity_desc = "Moderate Intensity / Steady Progression"
67          rest_time = "60 seconds"
68          cardio_time = "25-30 minutes moderate steady-state"
69          sets_reps = "3 sets of 10-12 reps"
70
71      is_weight_loss = "loss" in goal_norm or "cut" in goal_norm
72      is_muscle = "muscle" in goal_norm or "bulk" in goal_norm or "hypertrophy" in goal_norm
```

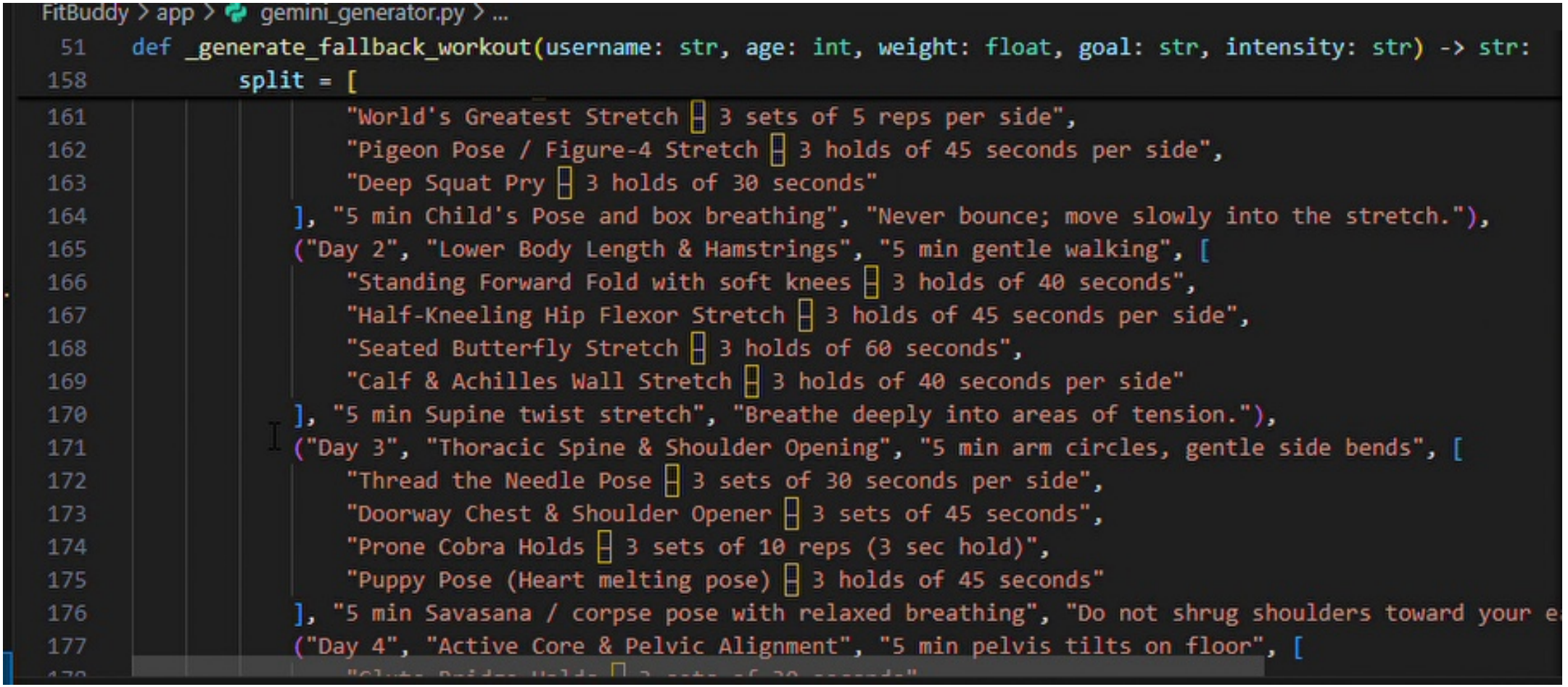

FitBuddy > app > gemini_generator.py > ...
def _generate_fallback_workout(username: str, age: int, weight: float, goal: str, intensity: str) -> str:
is_weight_loss = "loss" in goal_norm or "cut" in goal_norm
is_muscle = "muscle" in goal_norm or "bulk" in goal_norm or "hypertrophy" in goal_norm
is_strength = "strength" in goal_norm or "power" in goal_norm
is_flexibility = "flexib" in goal_norm or "mobility" in goal_norm or "yoga" in goal_norm
if is_muscle or is_strength:
split = [
("Day 1", "Upper Body Push & Core", "5 min arm circles, cat-cow, chest openers", [
f"Push-ups (or Incline Push-ups) — {sets_reps}, Rest: {rest_time}",
f"Dumbbell or Resistance Band Shoulder Press — {sets_reps}, Rest: {rest_time}",
f"Bodyweight Tricep Dips — {sets_reps}, Rest: {rest_time}",
f"Plank Hold — 3 sets of 30-45 seconds, Rest: 45 seconds"
], "5 min light shoulder & chest stretches", "Maintain a neutral spine and controlled eccentric
("Day 2", "Lower Body Focus (Quads & Glutes)", "5 min leg swings, bodyweight squats, hip opener
f"Bodyweight or Goblet Squats — {sets_reps}, Rest: {rest_time}",
f"Walking Lunges — 3 sets of 10 reps per leg, Rest: {rest_time}",
f"Glute Bridges — {sets_reps}, Rest: {rest_time}",
f"Standing Calf Raises — 3 sets of 15 reps, Rest: 45 seconds"

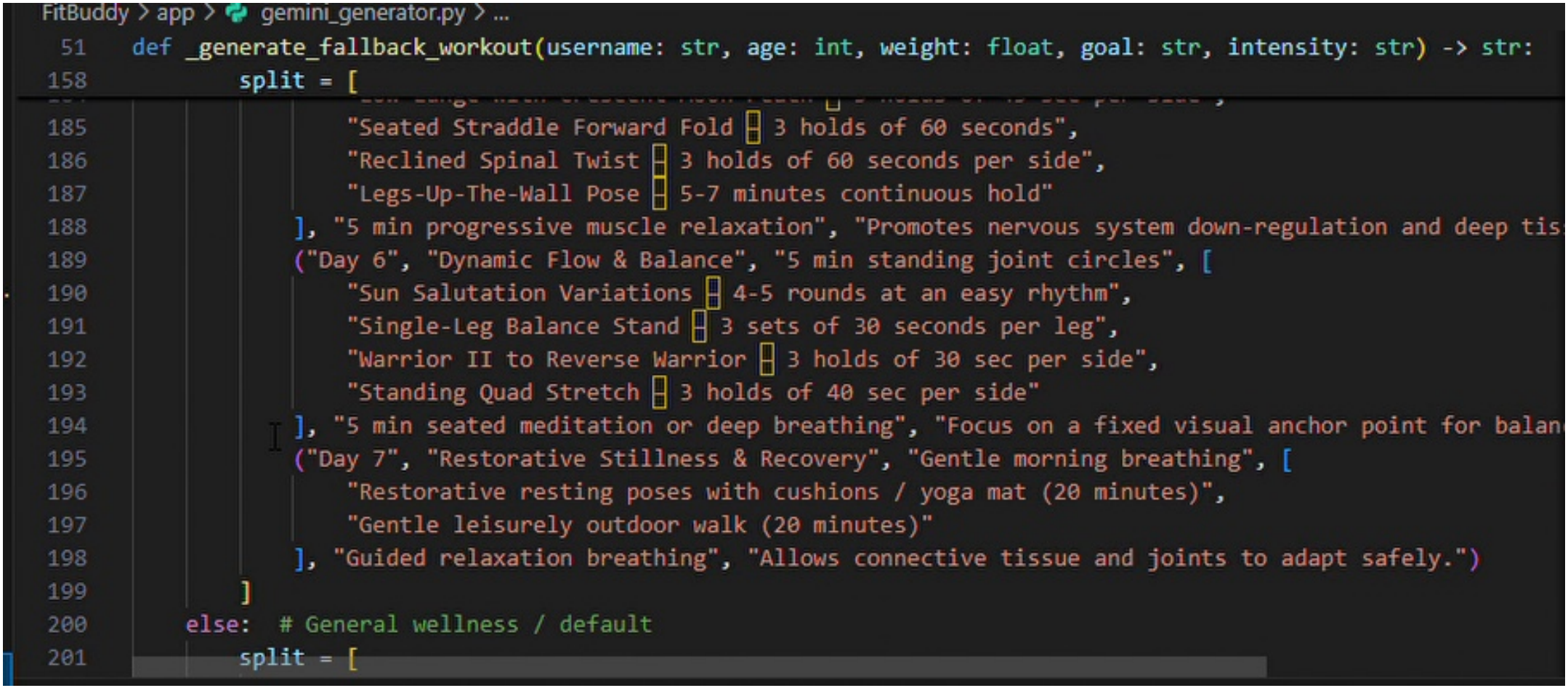

FitBuddy > app > gemini_generator.py > ...
51 def _generate_fallback_workout(username: str, age: int, weight: float, goal: str, intensity: str) -> str:
77 split = [
90 ("Day 3", "Active Recovery & Mobility", "5 min neck rolls and torso twists", [
91 "Full-body mobility routine: Cat-Cow, World's Greatest Stretch, Child's Pose (15 minutes)",
92 "20-30 minute brisk walk or gentle cycling"
93], "5 min deep diaphragmatic breathing", "Do not push into pain; promote circulation."),
94 ("Day 4", "Upper Body Pull & Posture", "5 min band pull-aparts, thoracic rotations", [
95 f"Dumbbell or Band Rows {sets_reps}, Rest: {rest_time}",
96 f"Lat Pulldowns or Band Pulldowns {sets_reps}, Rest: {rest_time}",
97 f"Bicep Curls {sets_reps}, Rest: {rest_time}",
98 f"Bird-Dog Holds 3 sets of 10 reps per side"
99], "5 min lat and upper back stretches", "Squeeze shoulder blades together at peak contraction.
100 ("Day 5", "Lower Body Posterior Chain & Core", "5 min hip circles, inchworms", [
101 f"Romanian Deadlifts (Dumbbells or Bands) {sets_reps}, Rest: {rest_time}",
102 f"Step-Ups 3 sets of 10 reps per leg, Rest: {rest_time}",
103 f"Side Plank Holds 3 sets of 25-35 seconds per side",
104 f"Bicycle Crunches 3 sets of 15 reps per side"
105], "5 min hamstring and glute foam rolling / stretching", "Hinge primarily at the hips; do not
106 ("Day 6", "Functional Full Body Conditioning", "5 min jumping jacks, light shadow boxing", [

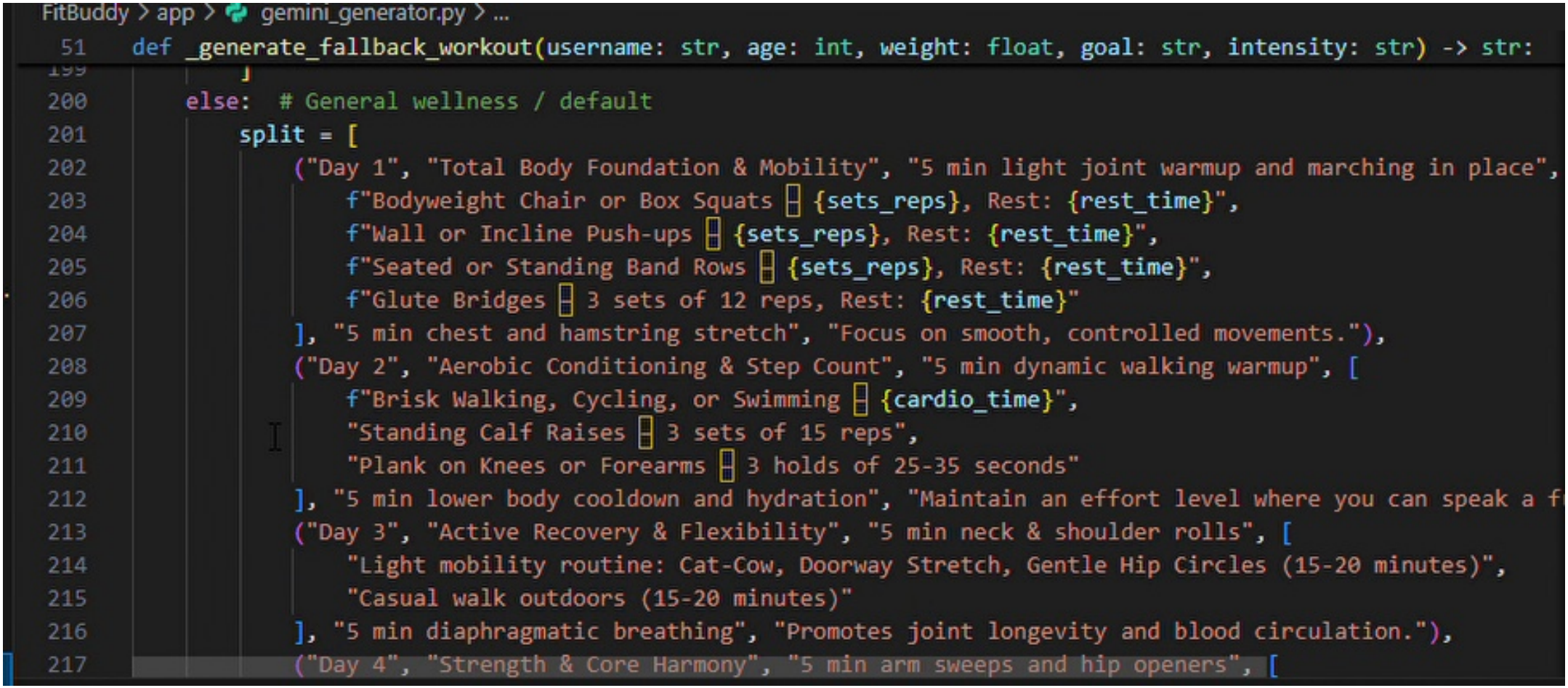
FitBuddy > app > gemini_generator.py > ...

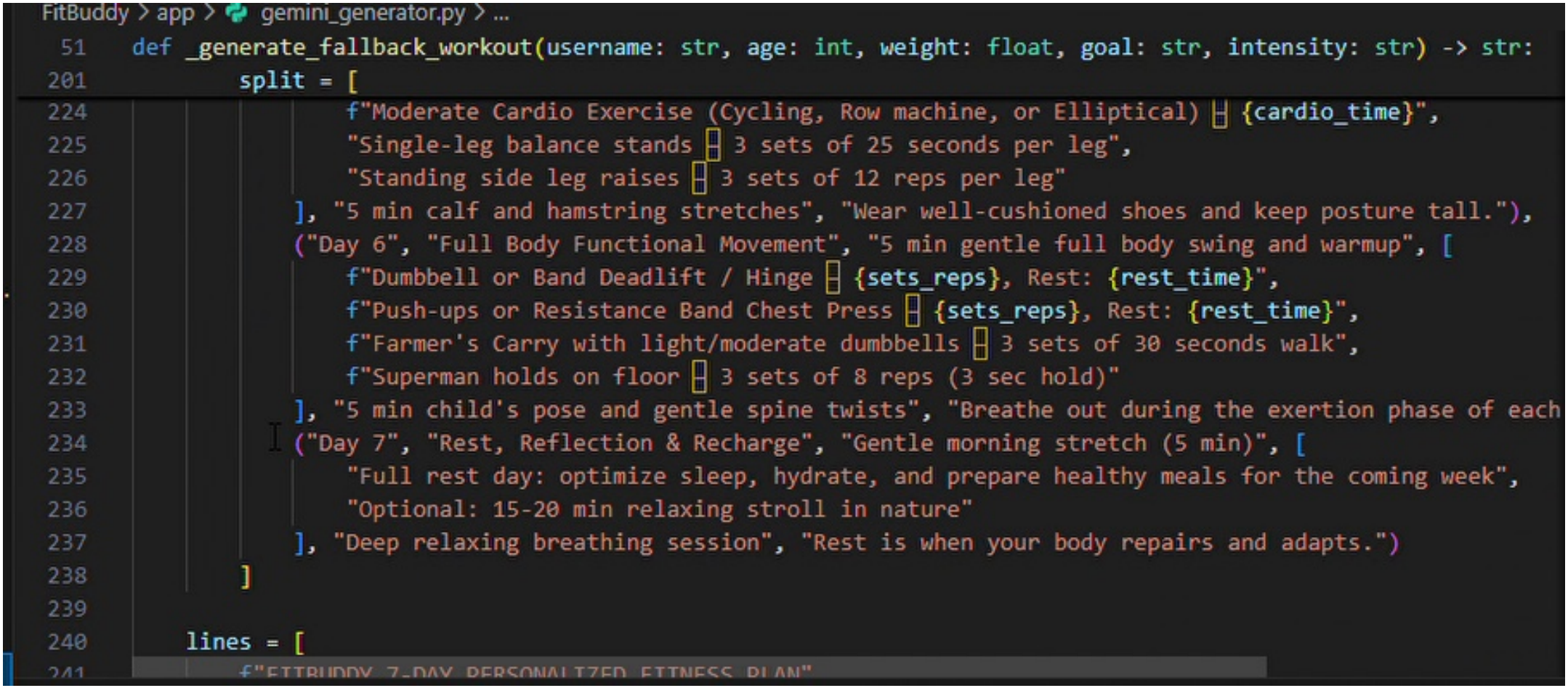
```
51  def _generate_fallback_workout(username: str, age: int, weight: float, goal: str, intensity: str) -> str:
77          split = [
111             ], "5 min full body cooldown and rehydration", "Prioritize form over speed."),
112             ("Day 7", "Complete Rest & Regeneration", "Gentle 5 min morning stretch", [
113                 "Total rest day: focus on 7-8 hours quality sleep, hydration, and wholesome nutrition",
114                 "Optional: 15-20 min relaxing casual stroll"
115             ], "Deep breathing and relaxation", "Rest allows muscle repair and glycogen replenishment.")
116         ]
117     elif is_weight_loss:
118         split = [
119             ("Day 1", "Full Body Circuit & Cardio Kickstart", "5 min dynamic stretching, light jumping jack
120                 f"Bodyweight Squats to Overhead Reach — {sets_reps}, Rest: {rest_time}",
121                 f"Modified Push-ups or Wall Push-ups — {sets_reps}, Rest: {rest_time}",
122                 f"Brisk Incline Walking or Cycling — {cardio_time}",
123                 f"Core Plank — 3 sets of 30-40 seconds"
124             ], "5 min leg and chest static stretches", "Stay hydrated and adjust pace if heart rate climbs
125             ("Day 2", "Cardio Conditioning & Core Stability", "5 min high knees, butt kicks, arm rotations"
126                 f"Low-Impact Cardio (Brisk walking, Elliptical, or Swimming) — {cardio_time}",
127                 f"Glute Bridges — {sets_reps}, Rest: {rest_time}",
```


FitBuddy > app > gemini_generator.py > ...
51 def _generate_fallback_workout(username: str, age: int, weight: float, goal: str, intensity: str) -> str:
118 split = [
140], "5 min upper body stretch", "Maintain continuous tension and steady breathing."),
141 ("Day 5", "Interval Cardio & Agility", "5 min light jog in place, ankle rolls", [
142 f"Interval Training (1 min brisk effort, 1 min recovery walk) 20-25 minutes",
143 f"Step-Ups on a low bench or stair 3 sets of 12 reps per leg",
144 f"Side Plank Holds 3 sets of 25 seconds per side"
145], "5 min calf, quad, and hip flexor stretches", "Wear supportive footwear and stay on level gr
146 ("Day 6", "Full Body Endurance & Core", "5 min arm sweeps, knee hugs", [
147 f"Goblet Squats or Chair Squats {sets_reps}, Rest: {rest_time}",
148 f"Lat Pulldowns or Band Pulls {sets_reps}, Rest: {rest_time}",
149 f"Steady-State Cardio (Power walk or cycling) 20 minutes",
150 f"Standard Plank 3 sets of 30-45 seconds"
151], "5 min full body cooldown", "Keep core engaged and posture upright."),
152 ("Day 7", "Rest, Hydration & Weekly Reset", "Gentle morning mobility (5 min)", [
153 "Full recovery day: light stretching, hydration target 2.5-3 liters water",
154 "Meal preparation and relaxation"
155], "Relaxation and breathing exercises", "Crucial for metabolic recovery and stress reduction."
156]


FitBuddy > app > gemini_generator.py > ...
51 def _generate_fallback_workout(username: str, age: int, weight: float, goal: str, intensity: str) -> str:
158 split = [
161 "World's Greatest Stretch – 3 sets of 5 reps per side",
162 "Pigeon Pose / Figure-4 Stretch – 3 holds of 45 seconds per side",
163 "Deep Squat Pry – 3 holds of 30 seconds"
164], "5 min Child's Pose and box breathing", "Never bounce; move slowly into the stretch."),
165 ("Day 2", "Lower Body Length & Hamstrings", "5 min gentle walking", [
166 "Standing Forward Fold with soft knees – 3 holds of 40 seconds",
167 "Half-Kneeling Hip Flexor Stretch – 3 holds of 45 seconds per side",
168 "Seated Butterfly Stretch – 3 holds of 60 seconds",
169 "Calf & Achilles Wall Stretch – 3 holds of 40 seconds per side"
170], "5 min Supine twist stretch", "Breathe deeply into areas of tension."),
171 ("Day 3", "Thoracic Spine & Shoulder Opening", "5 min arm circles, gentle side bends", [
172 "Thread the Needle Pose – 3 sets of 30 seconds per side",
173 "Doorway Chest & Shoulder Opener – 3 sets of 45 seconds",
174 "Prone Cobra Holds – 3 sets of 10 reps (3 sec hold)",
175 "Puppy Pose (Heart melting pose) – 3 holds of 45 seconds"
176], "5 min Savasana / corpse pose with relaxed breathing", "Do not shrug shoulders toward your e
177 ("Day 4", "Active Core & Pelvic Alignment", "5 min pelvis tilts on floor", [


FitBuddy > app > gemini_generator.py > ...
51 def _generate_fallback_workout(username: str, age: int, weight: float, goal: str, intensity: str) -> str:
158 split = [
185 "Seated Straddle Forward Fold – 3 holds of 60 seconds",
186 "Reclined Spinal Twist – 3 holds of 60 seconds per side",
187 "Legs-Up-The-Wall Pose – 5-7 minutes continuous hold"
188], "5 min progressive muscle relaxation", "Promotes nervous system down-regulation and deep tis
189 ("Day 6", "Dynamic Flow & Balance", "5 min standing joint circles", [
190 "Sun Salutation Variations – 4-5 rounds at an easy rhythm",
191 "Single-Leg Balance Stand – 3 sets of 30 seconds per leg",
192 "Warrior II to Reverse Warrior – 3 holds of 30 sec per side",
193 "Standing Quad Stretch – 3 holds of 40 sec per side"
194], "5 min seated meditation or deep breathing", "Focus on a fixed visual anchor point for balan
195 ("Day 7", "Restorative Stillness & Recovery", "Gentle morning breathing", [
196 "Restorative resting poses with cushions / yoga mat (20 minutes)",
197 "Gentle leisurely outdoor walk (20 minutes)"
198], "Guided relaxation breathing", "Allows connective tissue and joints to adapt safely.")
199]
200 else: # General wellness / default
201 split = [


FitBuddy > app > gemini_generator.py > ...
def _generate_fallback_workout(username: str, age: int, weight: float, goal: str, intensity: str) -> str:
]
else: # General wellness / default
split = [
("Day 1", "Total Body Foundation & Mobility", "5 min light joint warmup and marching in place",
f"Bodyweight Chair or Box Squats – {sets_reps}, Rest: {rest_time}",
f"Wall or Incline Push-ups – {sets_reps}, Rest: {rest_time}",
f"Seated or Standing Band Rows – {sets_reps}, Rest: {rest_time}",
f"Glute Bridges – 3 sets of 12 reps, Rest: {rest_time}"
], "5 min chest and hamstring stretch", "Focus on smooth, controlled movements."),
("Day 2", "Aerobic Conditioning & Step Count", "5 min dynamic walking warmup", [
f"Brisk Walking, Cycling, or Swimming – {cardio_time}",
"Standing Calf Raises – 3 sets of 15 reps",
"Plank on Knees or Forearms – 3 holds of 25-35 seconds"
], "5 min lower body cooldown and hydration", "Maintain an effort level where you can speak a f
("Day 3", "Active Recovery & Flexibility", "5 min neck & shoulder rolls", [
"Light mobility routine: Cat-Cow, Doorway Stretch, Gentle Hip Circles (15-20 minutes)",
"Casual walk outdoors (15-20 minutes)"
], "5 min diaphragmatic breathing", "Promotes joint longevity and blood circulation."),
("Day 4", "Strength & Core Harmony", "5 min arm sweeps and hip openers", [


FitBuddy > app > gemini_generator.py > ...
def _generate_fallback_workout(username: str, age: int, weight: float, goal: str, intensity: str) -> str:
split = [
f"Moderate Cardio Exercise (Cycling, Row machine, or Elliptical) – {cardio_time}",
"Single-leg balance stands – 3 sets of 25 seconds per leg",
"Standing side leg raises – 3 sets of 12 reps per leg"
], "5 min calf and hamstring stretches", "Wear well-cushioned shoes and keep posture tall."),
("Day 6", "Full Body Functional Movement", "5 min gentle full body swing and warmup", [
f"Dumbbell or Band Deadlift / Hinge – {sets_reps}, Rest: {rest_time}",
f"Push-ups or Resistance Band Chest Press – {sets_reps}, Rest: {rest_time}",
f"Farmer's Carry with light/moderate dumbbells – 3 sets of 30 seconds walk",
f"Superman holds on floor – 3 sets of 8 reps (3 sec hold)"
], "5 min child's pose and gentle spine twists", "Breathe out during the exertion phase of each
("Day 7", "Rest, Reflection & Recharge", "Gentle morning stretch (5 min)", [
"Full rest day: optimize sleep, hydrate, and prepare healthy meals for the coming week",
"Optional: 15-20 min relaxing stroll in nature"
], "Deep relaxing breathing session", "Rest is when your body repairs and adapts.")
]
lines = [
f"FITBUDDY 7-DAY PERSONALIZED FITNESS PLAN"

FitBuddy > app > gemini_generator.py > ...

```
51   def _generate_fallback_workout(username: str, age: int, weight: float, goal: str, intensity: str) -> str:
240      lines = [
245          ""
246      ]
247
248      for day_label, focus, warmup, exercises, cooldown, safety in split:
249          lines.append(f"{day_label}: {focus}")
250          lines.append(f"• Warm-up (5–10 min): {warmup}")
251          lines.append("• Main Workout:")
252          for ex in exercises:
253              lines.append(f"  - {ex}")
254          lines.append(f"• Cool-down & Recovery: {cooldown}")
255          lines.append(f"• Safety Note: {safety}")
256          lines.append("")
257
258      lines.append("General Safety Reminder: Listen to your body, stay well hydrated throughout every session
259      return "\n".join(lines)
260
261
262  def generate_workout_gemini(username, age, weight, goal, intensity):
```

FitBuddy > app > gemini_generator.py > ...

```python
def generate_workout_gemini(username, age, weight, goal, intensity):
    prompt = f"""
You are FitBuddy, a careful fitness-planning assistant.

Create a personalized 7-day workout plan for:
Name: {username}
Age: {age}
Weight: {weight} kg
Goal: {goal}
Preferred intensity: {intensity}

Return a clear plan for Day 1 through Day 7. For each day include:
- Focus
- Warm-up (5–10 minutes)
- Main workout with exercise names, sets/reps or duration, and rest
- Cool-down/recovery
- A brief safety note when appropriate

Make the plan realistic, progressive, and beginner-friendly unless the input clearly indicates otherwise.
```

FitBuddy > app > gemini_generator.py > ...

```
262    def generate_workout_gemini(username, age, weight, goal, intensity):
263        prompt = f"""
277    - Cool-down/recovery
278    - A brief safety note when appropriate
279
280    Make the plan realistic, progressive, and beginner-friendly unless the input clearly indicates otherwise.
281    Do not diagnose conditions, prescribe medication, or claim medical certainty.
282    If the user's age/weight/goal is insufficient to determine suitability, use conservative general guidance.
283    Use plain text headings and bullets. Do not use markdown tables.
284    """
285        # Attempt Gemini call if API key is present
286        if GOOGLE_API_KEY and not GOOGLE_API_KEY.startswith("AQ."):
287            try:
288                return _call_gemini(prompt)
289            except Exception as exc:
290                logger.warning("Gemini workout generation failed (%s), switching to intelligent fallback genera
291
292        # Safe fallback if Gemini is unavailable, unauthorized, or rate-limited
293        return _generate_fallback_workout(username, age, weight, goal, intensity)
```

FitBuddy > app > gemini_generator.py > ...

```
262    def generate_workout_gemini(username, age, weight, goal, intensity):
286        if GOOGLE_API_KEY and not GOOGLE_API_KEY.startswith("AQ."):
287            try:
288                return _call_gemini(prompt)
289            except Exception as exc:
290                logger.warning("Gemini workout generation failed (%s), switching to intelligent fallback genera
291
292        # Safe fallback if Gemini is unavailable, unauthorized, or rate-limited
293        return _generate_fallback_workout(username, age, weight, goal, intensity)
294
```

FitBuddy > app > main.py > ...

```python
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
from dotenv import load_dotenv

from .database import init_db
from .routes import router

BASE_DIR = Path(__file__).resolve().parent.parent
load_dotenv(BASE_DIR / ".env")

app = FastAPI(
    title="FitBuddy – AI Fitness Plan Generator",
    description="AI-powered 7-day fitness planning with feedback-based refinement.",
    version="1.0.0",
)

app.mount("/static", StaticFiles(directory=str(BASE_DIR / "static")), name="static")
templates = Jinja2Templates(directory=str(BASE_DIR / "templates"))

```

FitBuddy > app > main.py > ...

```python

@app.on_event("startup")
def startup():
    init_db()

app.include_router(router)

@app.get("/health")
def health():
    return {"status": "ok", "service": "FitBuddy"}

@app.exception_handler(404)
async def not_found(request: Request, exc):
    return templates.TemplateResponse(
        request=request,
        name="error.html",
        context={"code": 404, "message": "Page not found."},
        status_code=404,
    )

```

FitBuddy > app > main.py > ...

```python
def health():
    return {"status": "ok", "service": "FitBuddy"}

@app.exception_handler(404)
async def not_found(request: Request, exc):
    return templates.TemplateResponse(
        request=request,
        name="error.html",
        context={"code": 404, "message": "Page not found."},
        status_code=404,
    )
```

FitBuddy > app > routes.py > ...

```python
from .database import (
    save_user, save_plan, get_latest_plan, get_all_users_with_plans,
    update_plan, delete_user
)
from .gemini_generator import generate_workout_gemini
from .gemini_flash_generator import generate_nutrition_tip_with_flash
from .updated_plan import update_workout_plan
from .config import ADMIN_KEY

router = APIRouter()
from pathlib import Path
TEMPLATE_DIR = Path(__file__).resolve().parent.parent / "templates"
templates = Jinja2Templates(directory=str(TEMPLATE_DIR))

@router.get("/", response_class=HTMLResponse)
def home(request: Request):
    return templates.TemplateResponse(request=request, name="index.html", context={})

@router.post("/generate-workout", response_class=HTMLResponse)
def generate_workout(
```

FitBuddy > app > routes.py > ...

```python

router = APIRouter()
from pathlib import Path
TEMPLATE_DIR = Path(__file__).resolve().parent.parent / "templates"
templates = Jinja2Templates(directory=str(TEMPLATE_DIR))

@router.get("/", response_class=HTMLResponse)
def home(request: Request):
    return templates.TemplateResponse(request=request, name="index.html", context={})

@router.post("/generate-workout", response_class=HTMLResponse)
def generate_workout(
    request: Request,
    username: str = Form(...),
    user_id: str = Form(...),
    age: int = Form(...),
    weight: float = Form(...),
    goal: str = Form(...),
    intensity: str = Form(...),
):
```

FitBuddy > app > routes.py > ...

```
25    def generate_workout(
34        try:
35            data = UserInput(
36                username=username, user_id=user_id, age=age, weight=weight,
37                goal=goal, intensity=intensity
38            )
39            user = save_user(data)
40            workout = generate_workout_gemini(
41                data.username, data.age, data.weight, data.goal, data.intensity
42            )
43            tip = generate_nutrition_tip_with_flash(data.goal, data.age, data.weight)
44            plan = save_plan(user.id, workout, tip)
45            return templates.TemplateResponse(
46                request=request,
47                name="result.html",
48                context={
49                    "user": user, "plan": plan,
50                    "display_plan": plan.original_plan, "is_updated": False,
51                    "error": None
```

FitBuddy > app > routes.py > ...

```
25    def generate_workout(
55            return templates.TemplateResponse(
56                request=request,
57                name="index.html",
58                context={"error": f"Could not generate the plan: {exc}"},
59                status_code=500,
60            )
61
62    @router.post("/submit-feedback", response_class=HTMLResponse)
63    def submit_feedback(
64        request: Request,
65        user_id: str = Form(...),
66        feedback: str = Form(...),
67    ):
68        try:
69            data = FeedbackRequest(user_id=user_id, feedback=feedback)
70            user, plan = get_latest_plan(data.user_id)
71            if not user or not plan:
72                return templates.TemplateResponse(
```

FitBuddy > app > routes.py > ...

```
63    def submit_feedback(
73                    request=request,
74                    name="error.html",
75                    context={"code": 404, "message": "User or workout plan not found."},
76                    status_code=404,
77                )
78            revised = update_workout_plan(
79                plan.original_plan, data.feedback, user.goal, user.intensity
80            )
81            saved = update_plan(plan.id, revised, data.feedback)
82            return templates.TemplateResponse(
83                request=request,
84                name="result.html",
85                context={
86                    "user": user, "plan": saved,
87                    "display_plan": saved.updated_plan, "is_updated": True,
88                    "error": None
89                },
90            )
91        except Exception as exc:
```

FitBuddy > app > routes.py > ...

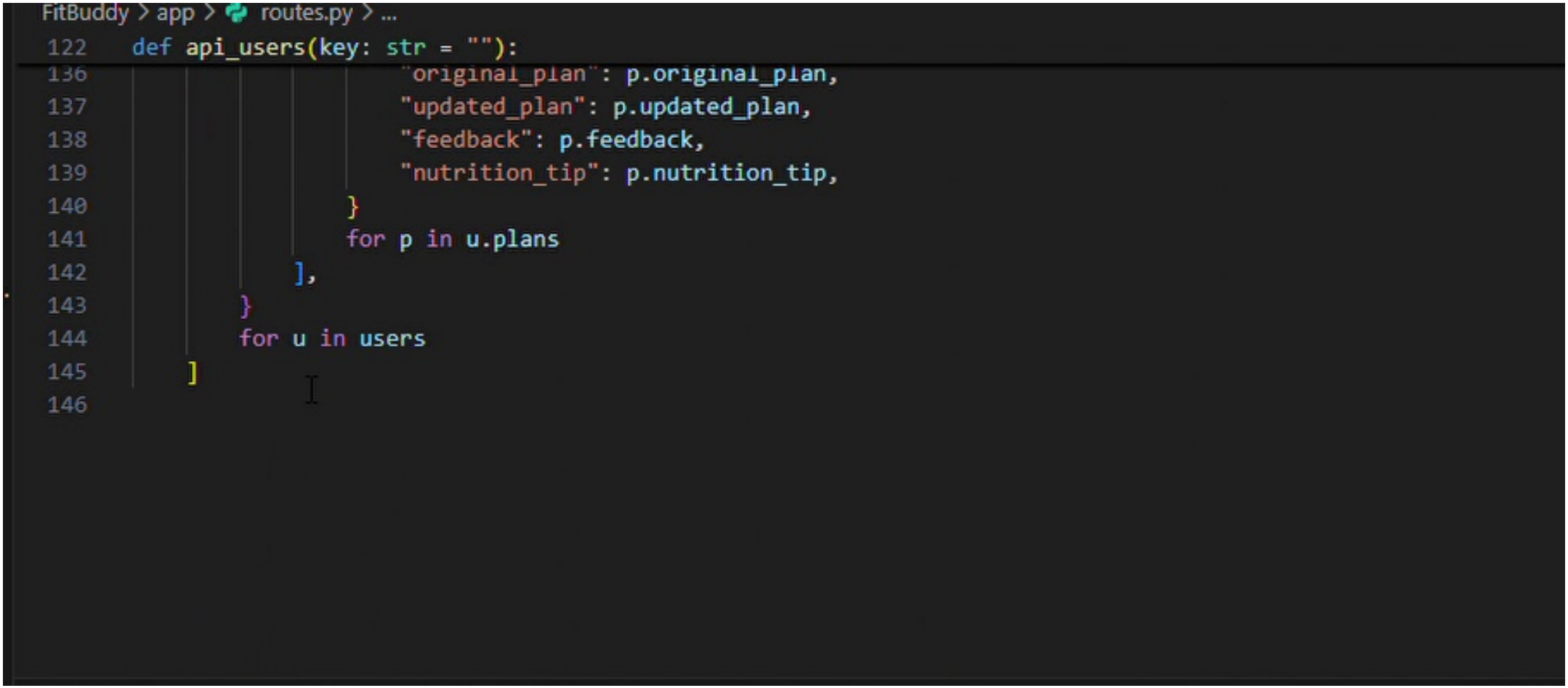
```
63    def submit_feedback(
86                    "user": user, "plan": saved,
87                    "display_plan": saved.updated_plan, "is_updated": True,
88                    "error": None
89                },
90            )
91        except Exception as exc:
92            return templates.TemplateResponse(
93                request=request,
94                name="error.html",
95                context={"code": 500, "message": f"Could not update the plan: {exc}"},
96                status_code=500,
97            )
98
99    @router.get("/view-all-users", response_class=HTMLResponse)
100   def view_all_users(request: Request, key: str = ""):
101       if key != ADMIN_KEY:
102           return templates.TemplateResponse(
103               request=request,
104               name="admin_login.html",
```

FitBuddy > app > routes.py > ...

```
63    def submit_feedback(
97        )
98
99    @router.get("/view-all-users", response_class=HTMLResponse)
100   def view_all_users(request: Request, key: str = ""):
101       if key != ADMIN_KEY:
102           return templates.TemplateResponse(
103               request=request,
104               name="admin_login.html",
105               context={"error": None}
106           )
107       users = get_all_users_with_plans()
108       return templates.TemplateResponse(
109           request=request,
110           name="all_users.html",
111           context={"users": users, "admin_key": key}
112       )
113
114   @router.post("/delete-user")
```

FitBuddy > app > routes.py > ...

```
119         return RedirectResponse(url=f"/view-all-users?key={key}", status_code=303)
120
121     @router.get("/api/users")
122     def api_users(key: str = ""):
123         if key != ADMIN_KEY:
124             return {"error": "Unauthorized"}
125         users = get_all_users_with_plans()
126         return [
127             {
128                 "user_id": u.user_id,
129                 "username": u.username,
130                 "age": u.age,
131                 "weight": u.weight,
132                 "goal": u.goal,
133                 "intensity": u.intensity,
134                 "plans": [
135                     {
136                         "original_plan": p.original_plan,
137                         "updated_plan": p.updated_plan,
138                         "feedback": p.feedback,
```


FitBuddy > app > routes.py > ...
def api_users(key: str = ""):
"original_plan": p.original_plan,
"updated_plan": p.updated_plan,
"feedback": p.feedback,
"nutrition_tip": p.nutrition_tip,
}
for p in u.plans
],
}
for u in users
]

FitBuddy > app > schemas.py > ...

```python
class UserInput(BaseModel):
    username: str = Field(min_length=1, max_length=120)
    user_id: str = Field(min_length=1, max_length=100)
    age: int = Field(ge=13, le=100)
    weight: float = Field(gt=20, lt=400)
    goal: str = Field(min_length=2, max_length=100)
    intensity: Literal["low", "medium", "high"]

    @field_validator("username", "user_id", "goal")
    @classmethod
    def clean_text(cls, value):
        return value.strip()

class FeedbackRequest(BaseModel):
    user_id: str = Field(min_length=1, max_length=100)
    feedback: str = Field(min_length=3, max_length=2000)
```

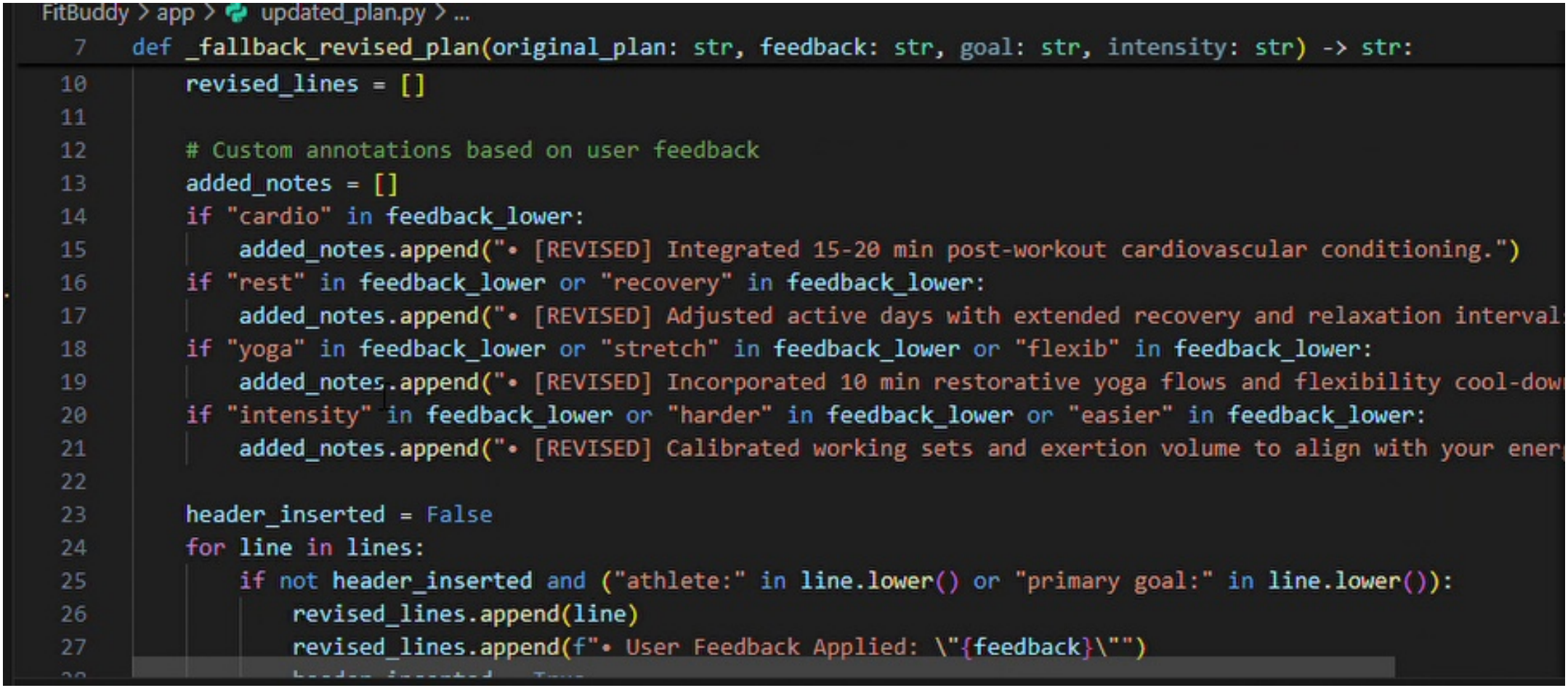
FitBuddy > app > schemas.py > ...

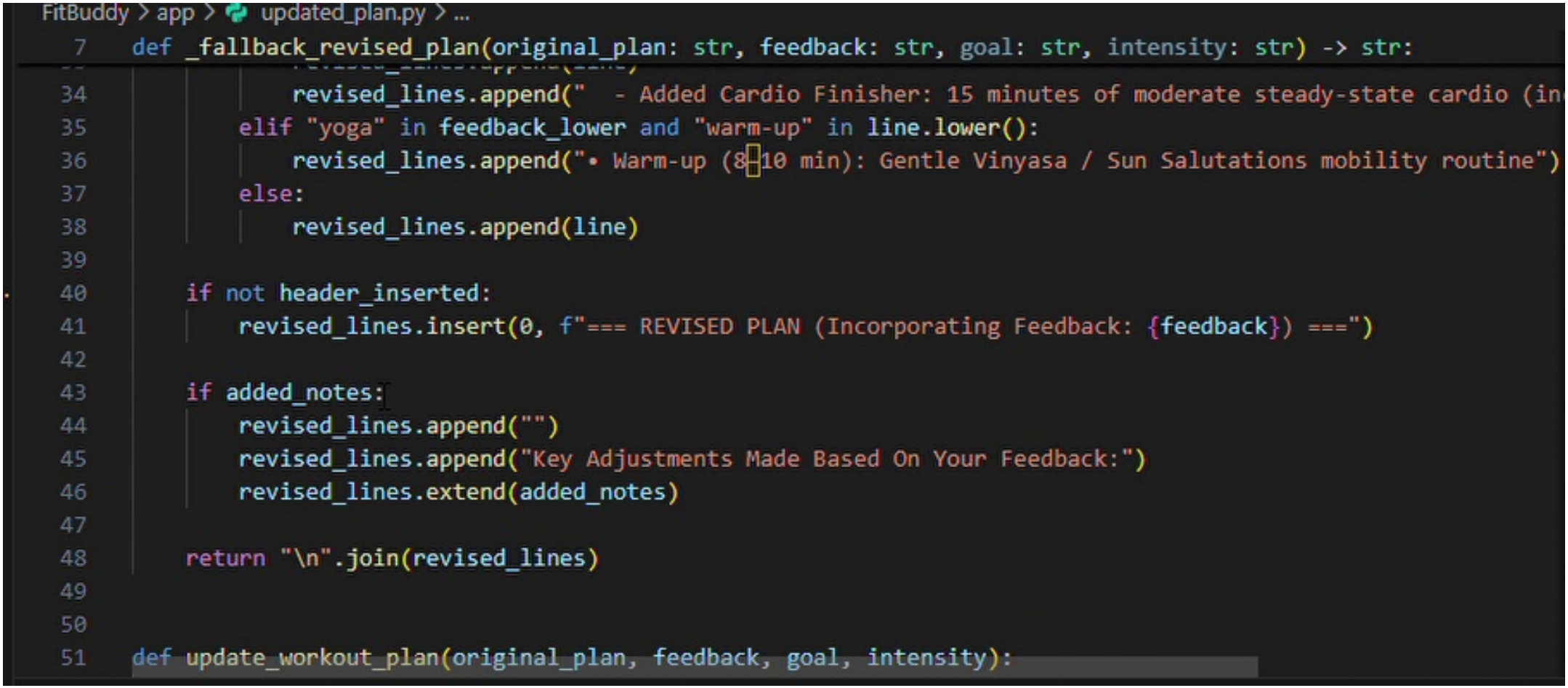
```python
class UserInput(BaseModel):
    intensity: Literal["low", "medium", "high"]

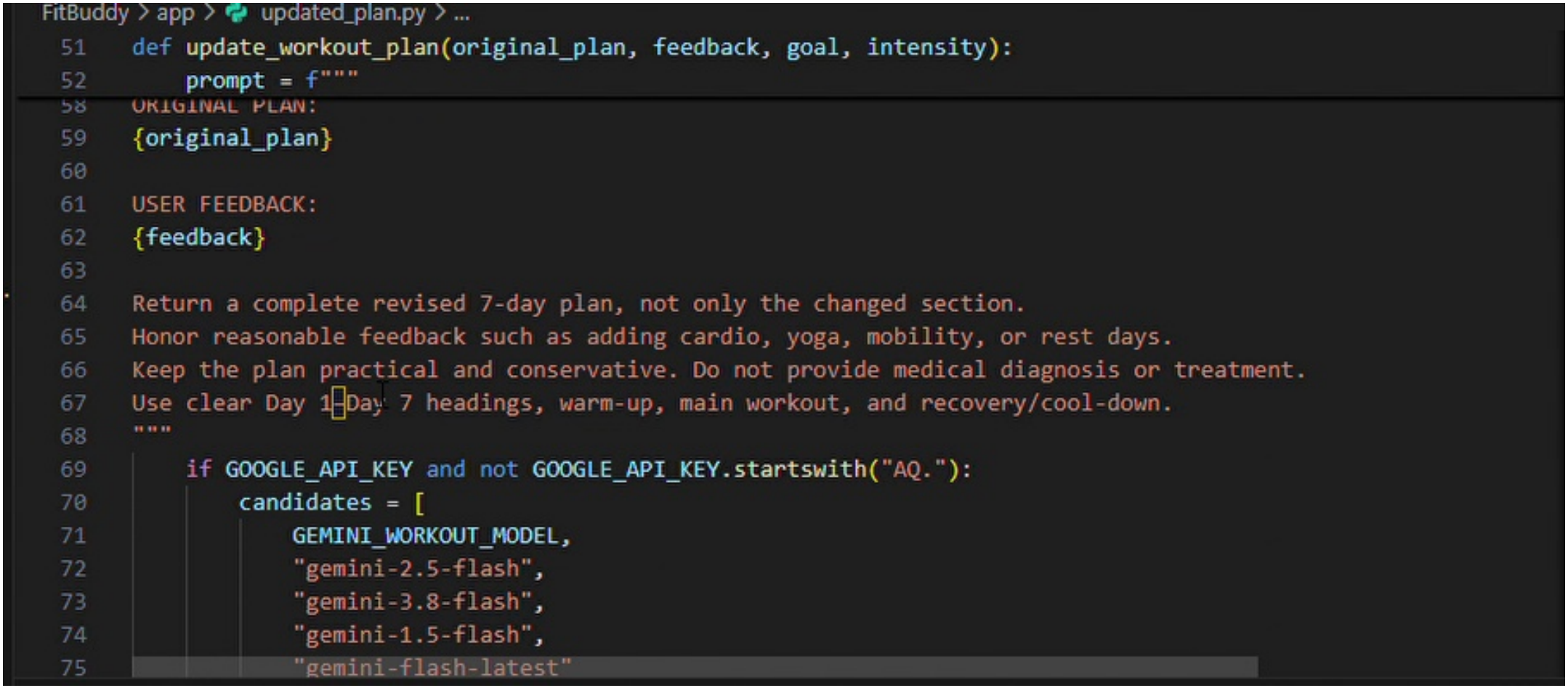
    @field_validator("username", "user_id", "goal")
    @classmethod
    def clean_text(cls, value):
        return value.strip()

class FeedbackRequest(BaseModel):
    user_id: str = Field(min_length=1, max_length=100)
    feedback: str = Field(min_length=3, max_length=2000)

```


FitBuddy > app > updated_plan.py > ...
def _fallback_revised_plan(original_plan: str, feedback: str, goal: str, intensity: str) -> str:
revised_lines = []
Custom annotations based on user feedback
added_notes = []
if "cardio" in feedback_lower:
added_notes.append("• [REVISED] Integrated 15-20 min post-workout cardiovascular conditioning.")
if "rest" in feedback_lower or "recovery" in feedback_lower:
added_notes.append("• [REVISED] Adjusted active days with extended recovery and relaxation interval
if "yoga" in feedback_lower or "stretch" in feedback_lower or "flexib" in feedback_lower:
added_notes.append("• [REVISED] Incorporated 10 min restorative yoga flows and flexibility cool-dow
if "intensity" in feedback_lower or "harder" in feedback_lower or "easier" in feedback_lower:
added_notes.append("• [REVISED] Calibrated working sets and exertion volume to align with your ener
header_inserted = False
for line in lines:
if not header_inserted and ("athlete:" in line.lower() or "primary goal:" in line.lower()):
revised_lines.append(line)
revised_lines.append(f"• User Feedback Applied: \"{feedback}\"")


FitBuddy > app > updated_plan.py > ...
7 def _fallback_revised_plan(original_plan: str, feedback: str, goal: str, intensity: str) -> str:
34 revised_lines.append(" - Added Cardio Finisher: 15 minutes of moderate steady-state cardio (in
35 elif "yoga" in feedback_lower and "warm-up" in line.lower():
36 revised_lines.append("• Warm-up (8–10 min): Gentle Vinyasa / Sun Salutations mobility routine")
37 else:
38 revised_lines.append(line)
39
40 if not header_inserted:
41 revised_lines.insert(0, f"=== REVISED PLAN (Incorporating Feedback: {feedback}) ===")
42
43 if added_notes:
44 revised_lines.append("")
45 revised_lines.append("Key Adjustments Made Based On Your Feedback:")
46 revised_lines.extend(added_notes)
47
48 return "\n".join(revised_lines)
49
50
51 def update_workout_plan(original_plan, feedback, goal, intensity):


FitBuddy > app > updated_plan.py > ...
51 def update_workout_plan(original_plan, feedback, goal, intensity):
52 prompt = f"""
58 ORIGINAL PLAN:
59 {original_plan}
60
61 USER FEEDBACK:
62 {feedback}
63
64 Return a complete revised 7-day plan, not only the changed section.
65 Honor reasonable feedback such as adding cardio, yoga, mobility, or rest days.
66 Keep the plan practical and conservative. Do not provide medical diagnosis or treatment.
67 Use clear Day 1-Day 7 headings, warm-up, main workout, and recovery/cool-down.
68 """
69 if GOOGLE_API_KEY and not GOOGLE_API_KEY.startswith("AQ."):
70 candidates = [
71 GEMINI_WORKOUT_MODEL,
72 "gemini-2.5-flash",
73 "gemini-3.8-flash",
74 "gemini-1.5-flash",
75 "gemini-flash-latest"

FitBuddy > app > updated_plan.py > ...

```
51    def update_workout_plan(original_plan, feedback, goal, intensity):
70            candidates = [
74                "gemini-1.5-flash",
75                "gemini-flash-latest"
76            ]
77            seen = set()
78            for model_name in candidates:
79                if not model_name or model_name in seen:
80                    continue
81                seen.add(model_name)
82                client = None
83                try:
84                    client = genai.Client(api_key=GOOGLE_API_KEY)
85                    response = client.models.generate_content(
86                        model=model_name,
87                        contents=prompt,
88                    )
89                    if response and response.text and response.text.strip():
90                        return response.text.strip()
91                except Exception as exc:
```

FitBuddy > app > updated_plan.py > ...

```
51  def update_workout_plan(original_plan, feedback, goal, intensity):
92                  logger.warning("Gemini revision model %s failed: %s", model_name, exc)
93                  continue
94              finally:
95                  if client is not None:
96                      try:
97                          if hasattr(client, "_api_client") and hasattr(client._api_client, "_httpx_client"):
98                              client._api_client._httpx_client.close()
99                      except Exception:
100                         pass
101
102     return _fallback_revised_plan(original_plan, feedback, goal, intensity)
103
```

FitBuddy > app > updated_plan.py > ...

```
51  def update_workout_plan(original_plan, feedback, goal, intensity):
94                  finally:
95                      if client is not None:
96                          try:
97                              if hasattr(client, "_api_client") and hasattr(client._api_client, "_httpx_client"):
98                                  client._api_client._httpx_client.close()
99                          except Exception:
100                             pass
101
102     return _fallback_revised_plan(original_plan, feedback, goal, intensity)
103
```

FitBuddy > templates > admin_login.html > ...

```
{% extends "base.html" %}
{% block title %}FitBuddy – Admin{% endblock %}
{% block content %}
<section class="center-page">
  <div class="card form-card small">
    <div class="pill">ADMIN DASHBOARD</div>
    <h1>Enter admin key</h1>
    <form action="/view-all-users" method="get">
      <input name="key" type="password" required placeholder="Admin key">
      <button class="primary" type="submit">Open Dashboard →</button>
    </form>
  </div>
</section>
{% endblock %}

```

FitBuddy > templates > admin_login.html > ...

```
4   <section class="center-page">
5     <div class="card form-card small">
8       <form action="/view-all-users" method="get">
9         <input name="key" type="password" required placeholder="Admin Key" >
10        <button class="primary" type="submit">Open Dashboard →</button>
11      </form>
12    </div>
13  </section>
14  {% endblock %}
15
```

FitBuddy > templates > all_users.html > ...

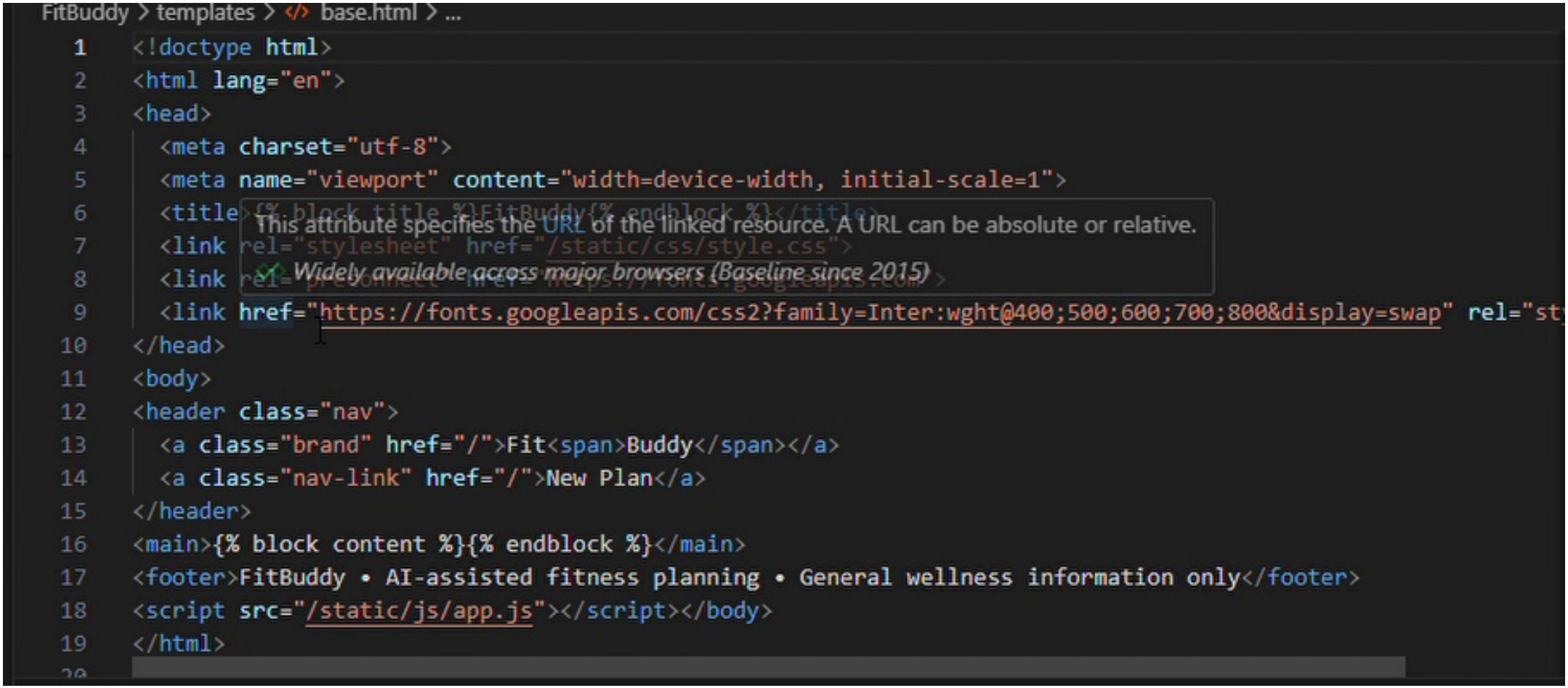
```
<section class="page">
  <div class="result-head">
    <div><div class="pill">ADMIN</div><h1>Users & Plans</h1><p>Review stored user profiles and plan revisio
    <a class="secondary" href="/">Home</a>
  </div>
  <div class="admin-list">
  {% for user in users %}
    <article class="card admin-card">
      <div class="admin-top">
        <div><h2>{{ user.username }}</h2><p>{{ user.user_id }} • {{ user.age }} yrs • {{ user.weight }} kg
        <form action="/delete-user" method="post" onsubmit="return confirm('Delete this user and plans?')">
          <input type="hidden" name="user_id" value="{{ user.user_id }}">
          <input type="hidden" name="key" value="{{ admin_key }}">
          <button class="danger" type="submit">Delete</button>
        </form>
      </div>
      {% for plan in user.plans %}
        <details>
          <summary>Plan #{{ plan.id }} — {{ plan.created_at.strftime('%Y-%m-%d %H:%M') if plan.created_at e
```

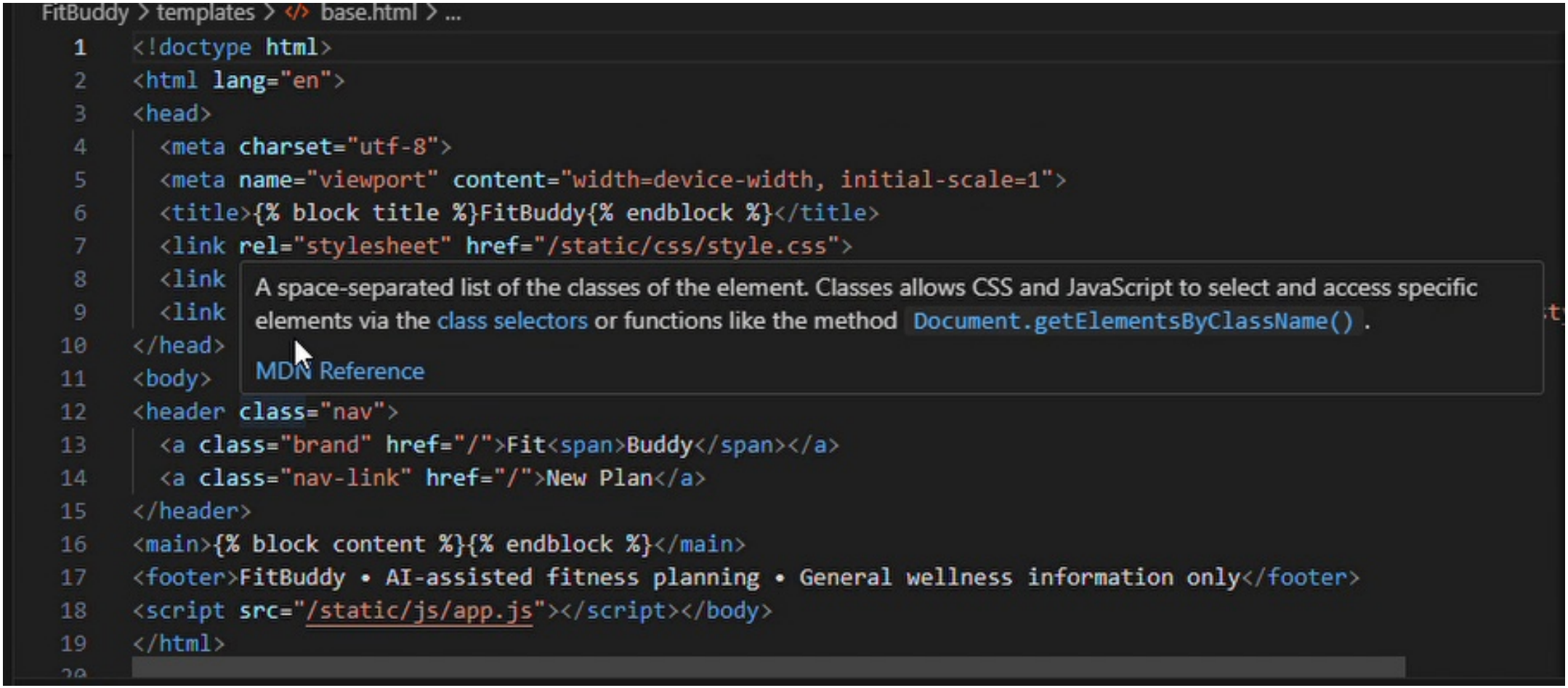
FitBuddy > templates > all_users.html > ...

```
 4   <section class="page">
 9     <div class="admin-list">
11       <article class="card admin-card">
20         {% for plan in user.plans %}
21           <details>
22             <summary>Plan #{{ plan.id }} — {{ plan.created_at.strftime('%Y-%m-%d %H:%M') if plan.created_at e
23             <h4>Original</h4><pre class="compact">{{ plan.original_plan }}</pre>
24             {% if plan.updated_plan %}<h4>Updated</h4><pre class="compact">{{ plan.updated_plan }}</pre>{% en
25             {% if plan.feedback %}<h4>Feedback</h4><p>{{ plan.feedback }}</p>{% endif %}
26           </details>
27         {% endfor %}
28       </article>
29     {% else %}
30       <div class="card empty">No users have been registered yet.</div>
31     {% endfor %}
32     </div>
33   </section>
34   {% endblock %}
35
```

FitBuddy > templates > all_users.html > ...

```
4   <section class="page">
9     <div class="admin-list">
11      <article class="card admin-card">
21          <details>
24            {% if plan.updated_plan %}<h4>Updated</h4><pre class="compact">{{ plan.updated_plan }}</pre>{% en
25            {% if plan.feedback %}<h4>Feedback</h4><p>{{ plan.feedback }}</p>{% endif %}
26          </details>
27        {% endfor %}
28      </article>
29    {% else %}
30      <div class="card empty">No users have been registered yet.</div>
31    {% endfor %}
32    </div>
33  </section>
34  {% endblock %}
35
```


FitBuddy > templates > base.html > ...
1 <!doctype html>
2 <html lang="en">
3 <head>
4 <meta charset="utf-8">
5 <meta name="viewport" content="width=device-width, initial-scale=1">
6 <title>{% block title %}FitBuddy{% endblock %}</title>
7 <link rel="stylesheet" href="/static/css/style.css">
8 <link rel="preconnect" href="https://fonts.googleapis.com">
9 <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700;800&display=swap" rel="st
10 </head>
11 <body>
12 <header class="nav">
13 FitBuddy
14 New Plan
15 </header>
16 <main>{% block content %}{% endblock %}</main>
17 <footer>FitBuddy • AI-assisted fitness planning • General wellness information only</footer>
18 <script src="/static/js/app.js"></script></body>
19 </html>
This attribute specifies the URL of the linked resource. A URL can be absolute or relative.
Widely available across major browsers (Baseline since 2015)


FitBuddy > templates > base.html > ...
1 <!doctype html>
2 <html lang="en">
3 <head>
4 <meta charset="utf-8">
5 <meta name="viewport" content="width=device-width, initial-scale=1">
6 <title>{% block title %}FitBuddy{% endblock %}</title>
7 <link rel="stylesheet" href="/static/css/style.css">
8 <link
9 <link
10 </head>
11 <body>
12 <header class="nav">
13 FitBuddy
14 New Plan
15 </header>
16 <main>{% block content %}{% endblock %}</main>
17 <footer>FitBuddy • AI-assisted fitness planning • General wellness information only</footer>
18 <script src="/static/js/app.js"></script></body>
19 </html>
A space-separated list of the classes of the element. Classes allows CSS and JavaScript to select and access specific elements via the class selectors or functions like the method Document.getElementsByClassName() .
MDN Reference

FitBuddy > templates > error.html > ...

```
{% extends "base.html" %}
{% block title %}FitBuddy – Error{% endblock %}
{% block content %}
<section class="center-page">
  <div class="card form-card small">
    <div class="pill">ERROR {{ code }}</div>
    <h1>Something went wrong</h1>
    <p>{{ message }}</p>
    <a class="primary button-link" href="/">Back to FitBuddy</a>
  </div>
</section>
{% endblock %}

```

FitBuddy > templates > error.html > ...

```
4   <section class="center-page">
5     <div class="card form-card small">
6       <div class="pill">ERROR {{ code }}</div>
7       <h1>Something went wrong</h1>
8       <p>{{ message }}</p>
9       <a class="primary button-link" href="/">Back to FitBuddy</a>
10    </div>
11  </section>
12  {% endblock %}
13
```

FitBuddy > templates > index.html > ...

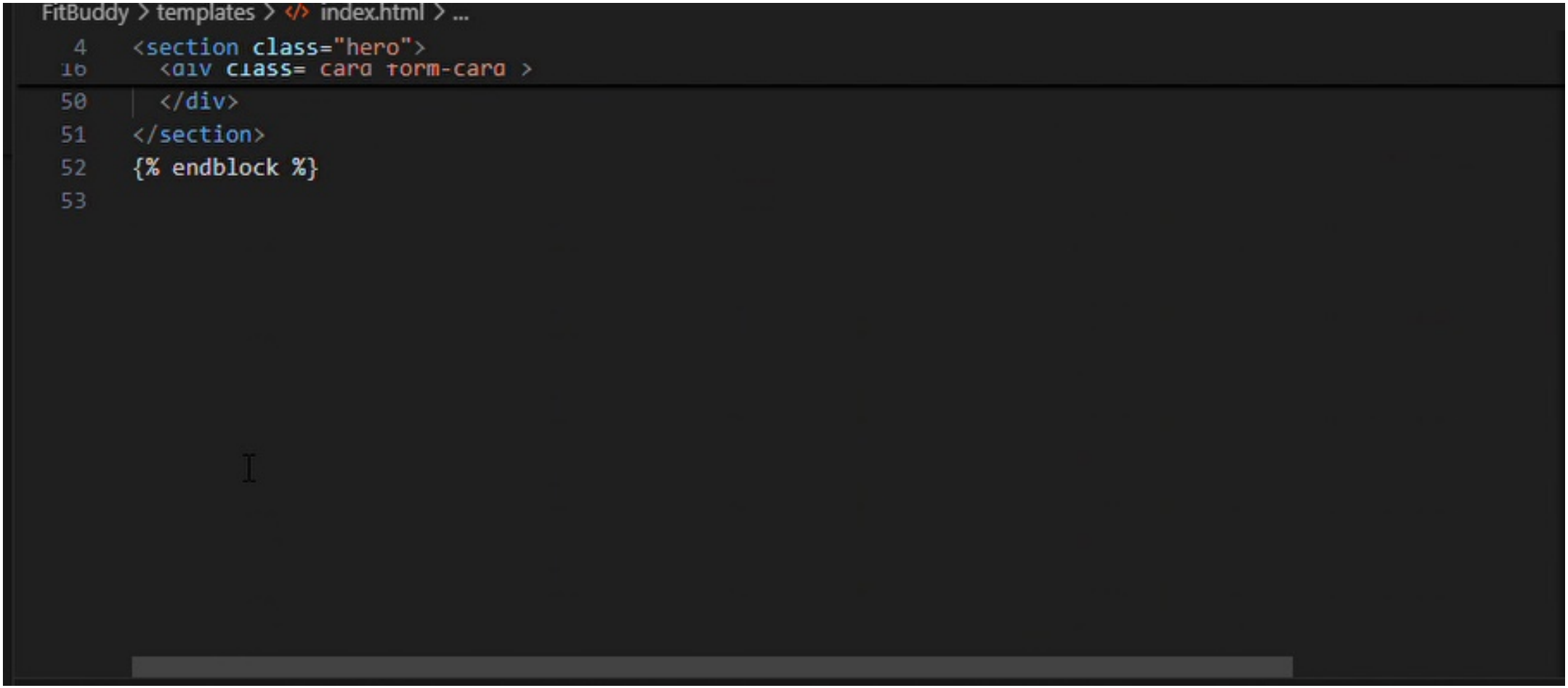
```
4   <section class="hero">
5     <div class="hero-copy">
8       <p>Generate a personalized 7-day workout plan and a practical nutrition or recovery tip with Gemini AI.
9       <div class="features">
10        <div><b>7</b><small>day plan</small></div>
11        <div><b>AI</b><small>adaptive</small></div>
12        <div><b>24/7</b><small>accessible</small></div>
13      </div>
14    </div>
15
16    <div class="card form-card">
17      <div class="card-heading">
18        <span class="icon">⚡</span>
19        <div><h2>Start your plan</h2><p>Tell FitBuddy a little about you.</p></div>
20      </div>
21      {% if error %}<div class="alert">{{ error }}</div>{% endif %}
22      <form action="/generate-workout" method="post">
23        <div class="grid-2">
24          <label>Name<input name="username" required maxlength="120" placeholder="Your name"></label>
25          <label>User ID<input name="user_id" required maxlength="100" placeholder="e.g. FB001"></label>
```

FitBuddy > templates > index.html > ...

```
 4   <section class="hero">
16     <div class="card form-card">
17       <div class="card-heading">
20       </div>
21       {% if error %}<div class="alert">{{ error }}</div>{% endif %}
22       <form action="/generate-workout" method="post">
23         <div class="grid-2">
24           <label>Name<input name="username" required maxlength="120" placeholder="Your name"></label>
25           <label>User ID<input name="user_id" required maxlength="100" placeholder="e.g. FB001"></label>
26           <label>Age<input type="number" name="age" min="13" max="100" required placeholder="21"></label>
27           <label>Weight (kg)<input type="number" name="weight" min="20" max="400" step="0.1" required placeho
28         </div>
29         <label>Fitness goal
30           <select name="goal" required>
31             <option value="">Choose a goal</option>
32             <option>Weight loss</option>
33             <option>Muscle gain</option>
34             <option>General wellness</option>
35             <option>Strength</option>
```

FitBuddy > templates > index.html > ...

```
 4   <section class="hero">
16     <div class="card form-card">
22       <form action="/generate-workout" method="post">
29         <label>Fitness goal
30           <select name="goal" required>
35             <option>Strength</option>
36             <option>Flexibility</option>
37             <option>Cardio fitness</option>
38           </select>
39         </label>
40         <label>Workout intensity
41           <div class="intensity">
42             <label><input type="radio" name="intensity" value="low" required><span>Low</span></label>
43             <label><input type="radio" name="intensity" value="medium"><span>Medium</span></label>
44             <label><input type="radio" name="intensity" value="high"><span>High</span></label>
45           </div>
46         </label>
47         <button class="primary" type="submit">Generate My Plan <span>→</span></button>
48       </form>
```


FitBuddy > templates > index.html > ...
4 <section class="hero">
16 <div class= card form-card >
50
51 </section>
52 {% endblock %}
53

FitBuddy > templates > result.html > ...

```
 4   <section class="page">
25     <div class="content-grid">
31       <aside>
44       </div>
45       </aside>
46     </div>
47   </section>
48   {% endblock %}
49
```

FitBuddy > templates > result.html > ...

```
<section class="page">
  <div class="result-head">
    <div>
      <div class="pill">✦ {% if is_updated %}UPDATED PLAN{% else %}YOUR PERSONAL PLAN{% endif %}</div>
      <h1>Ready, {{ user.username }}.</h1>
      <p>{{ user.goal }} • {{ user.intensity|capitalize }} intensity • {{ user.weight }} kg</p>
    </div>
    <a class="secondary" href="/">← New plan</a>
  </div>

  <div class="stats">
    <div><small>AGE</small><b>{{ user.age }}</b></div>
    <div><small>GOAL</small><b>{{ user.goal }}</b></div>
    <div><small>INTENSITY</small><b>{{ user.intensity|capitalize }}</b></div>
    <div><small>USER ID</small><b>{{ user.user_id }}</b></div>
  </div>

  {% if is_updated %}
  <div class="success">✓ Your feedback was applied and the revised plan was saved.</div>
```

FitBuddy > templates > result.html > ...

```
4   <section class="page">
5     <div class="result-head">
11      <a class="secondary" href="/">← New plan</a>
12    </div>
13
14    <div class="stats">
15      <div><small>AGE</small><b>{{ user.age }}</b></div>
16      <div><small>GOAL</small><b>{{ user.goal }}</b></div>
17      <div><small>INTENSITY</small><b>{{ user.intensity|capitalize }}</b></div>
18      <div><small>USER ID</small><b>{{ user.user_id }}</b></div>
19    </div>
20
21    {% if is_updated %}
22    <div class="success">✓ Your feedback was applied and the revised plan was saved.</div>
23    {% endif %}
24
25    <div class="content-grid">
26      <article class="card plan-card">
27        <div class="section-title"><span>🏋</span><div><h2>7-Day Workout Plan</h2><p>AI-generated routine tai
```

FitBuddy > templates > result.html > ...

```
 4   <section class="page">
25     <div class="content-grid">
31       <aside>
37         <div class="card feedback-card">
44         </div>
45       </aside>
46     </div>
47   </section>
48   {% endblock %}
49
```

FitBuddy > templates > result.html > ...

```
 4   <section class="page">
25     <div class="content-grid">
31       <aside>
44         </div>
45       </aside>
46     </div>
47   </section>
48   {% endblock %}
49
```

FitBuddy > .env

```
4   ADMIN_KEY=change-this-admin-key
5
```

FitBuddy > .env

```
4   ADMIN_KEY=change-this-admin-key
5
```

FitBuddy > .env.example

```
# Copy this file to .env and add your Gemini API key.
GOOGLE_API_KEY=your_gemini_api_key_here

# Models can be changed if your Google account/project exposes different models.
GEMINI_WORKOUT_MODEL=gemini-2.5-pro
GEMINI_TIP_MODEL=gemini-2.5-flash

# Admin dashboard password/key
ADMIN_KEY=change-this-admin-key

# Optional; default is a local SQLite database named fitbuddy.db
# DATABASE_URL=sqlite:///./fitbuddy.db

```

FitBuddy > .env.example

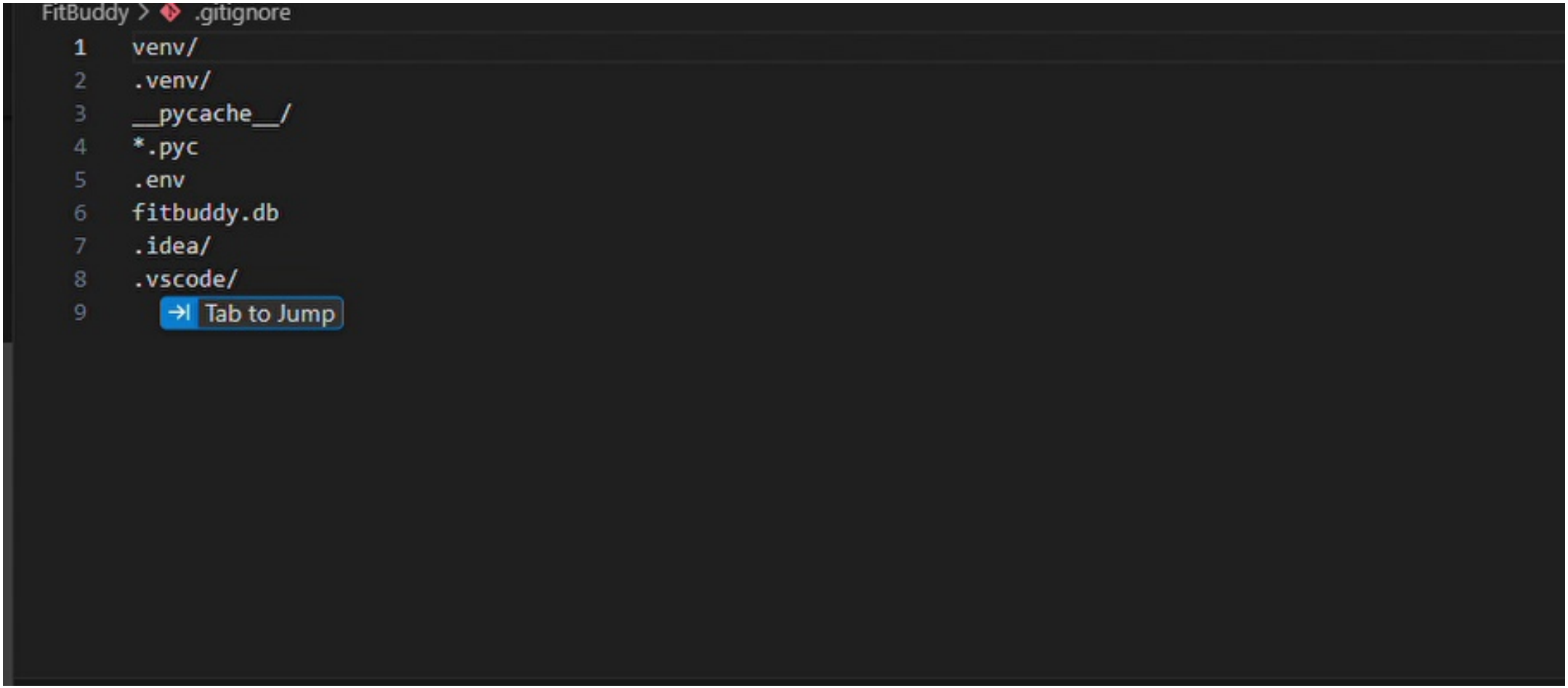
```
# Copy this file to .env and add your Gemini API key.
GOOGLE_API_KEY=your_gemini_api_key_here

# Models can be changed if your Google account/project exposes different models.
GEMINI_WORKOUT_MODEL=gemini-2.5-pro
GEMINI_TIP_MODEL=gemini-2.5-flash

# Admin dashboard password/key
ADMIN_KEY=change-this-admin-key

# Optional; default is a local SQLite database named fitbuddy.db
# DATABASE_URL=sqlite:///./fitbuddy.db

```


FitBuddy > .gitignore
1 venv/
2 .venv/
3 __pycache__/
4 *.pyc
5 .env
6 fitbuddy.db
7 .idea/
8 .vscode/
9
Tab to Jump

FitBuddy > requirements.txt

```
fastapi>=0.115
uvicorn[standard]>=0.30
jinja2>=3.1
sqlalchemy>=2.0
python-multipart>=0.0.9
python-dotenv>=1.0
google-genai>=1.0
pydantic>=2.7

```

FitBuddy > requirements.txt

```
fastapi>=0.115
uvicorn[standard]>=0.30
jinja2>=3.1
sqlalchemy>=2.0
python-multipart>=0.0.9
python-dotenv>=1.0
google-genai>=1.0
pydantic>=2.7

```

FitBuddy > requirements.txt

```
6   python-dotenv>=1.0
7   google-genai>=1.0
8   pydantic>=2.7
9
```

FitBuddy > requirements.txt

```
6   python-dotenv>=1.0
7   google-genai>=1.0
8   pydantic>=2.7
9
```